



Universidad Nacional Autónoma de México
Facultad de Ingeniería

Sistemas Distribuidos — Semestre 2027-1

Laboratorio S09: Que el aviso no tumbe la transferencia

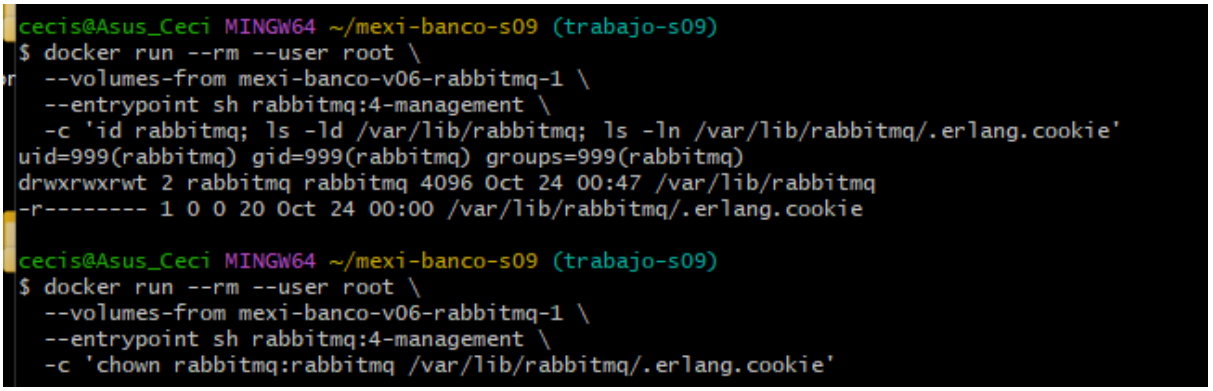
Alumna: Cecilia Ximena Solís Cisneros
Fecha de entrega: 23 de octubre de 2026

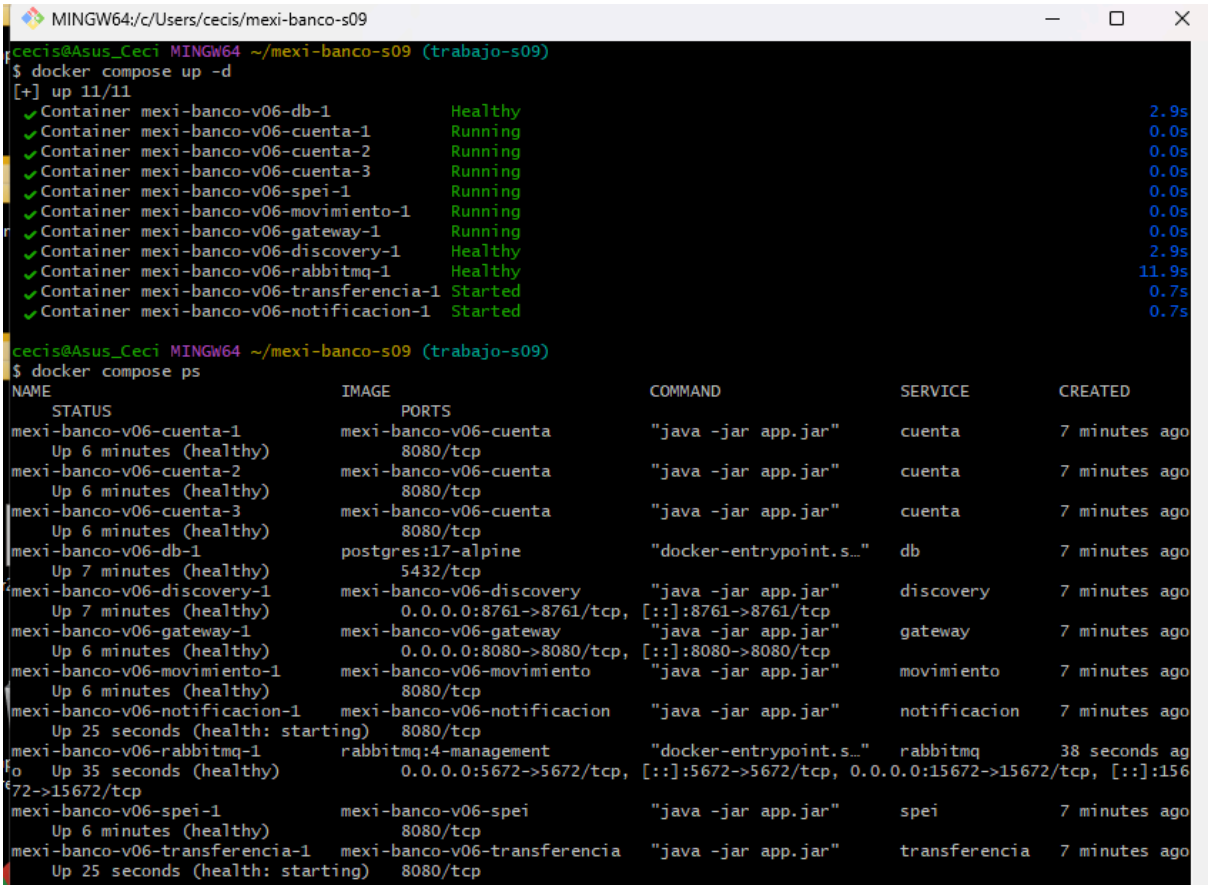
Objetivo
Al terminar, la transferencia de Mexi Banco avisa con un evento en RabbitMQ en lugar de llamar por HTTP a notificación, y tu consumidor sobrevive a una caída, a un duplicado y a un mensaje envenenado sin perder ni repetir avisos.

Desarrollo

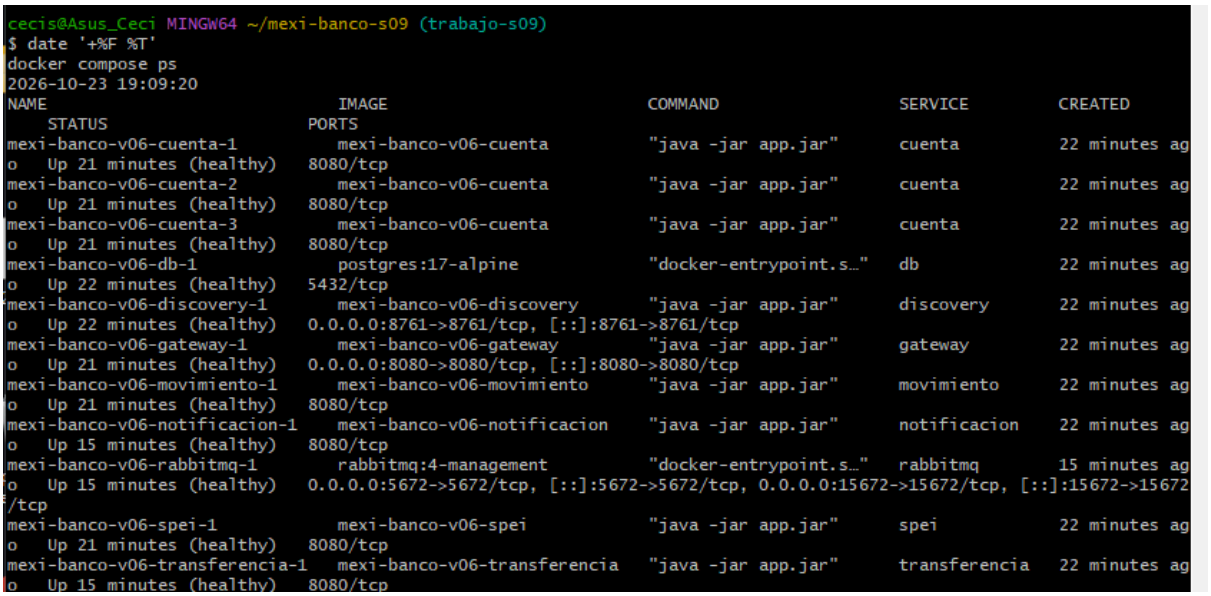
Descargué la rama 05-mensajeria-base y creé la rama de trabajo trabajo-s09. Ajusté los finales de línea de los archivos mvnw a LF para construir los servicios en Docker.

RabbitMQ falló inicialmente porque no podía leer su archivo .erlang.cookie. Al revisar los permisos, encontré que pertenecía a root y solo su propietario podía leerlo. Cambié el propietario al usuario rabbitmq y volví a iniciar los servicios. Después comprobé que RabbitMQ estaba healthy.



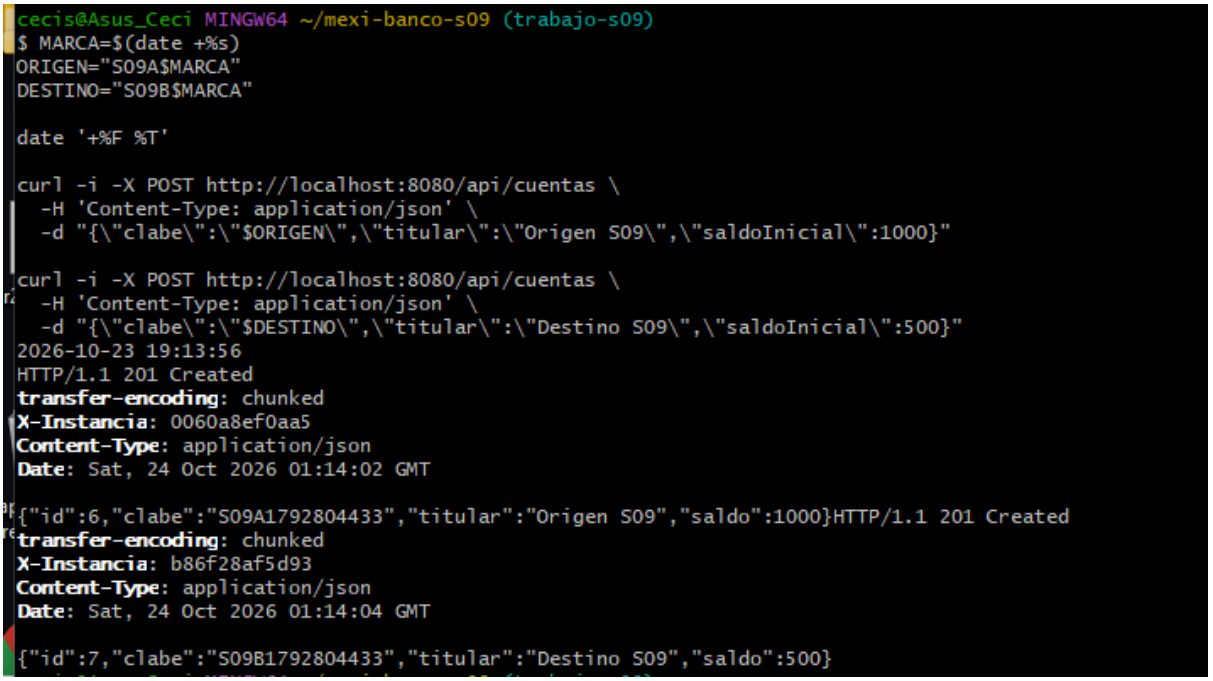


El 23 de octubre de 2026, a las 19:09:20, comprobé que los once contenedores estaban healthy, incluido RabbitMQ y las tres copias de cuenta.



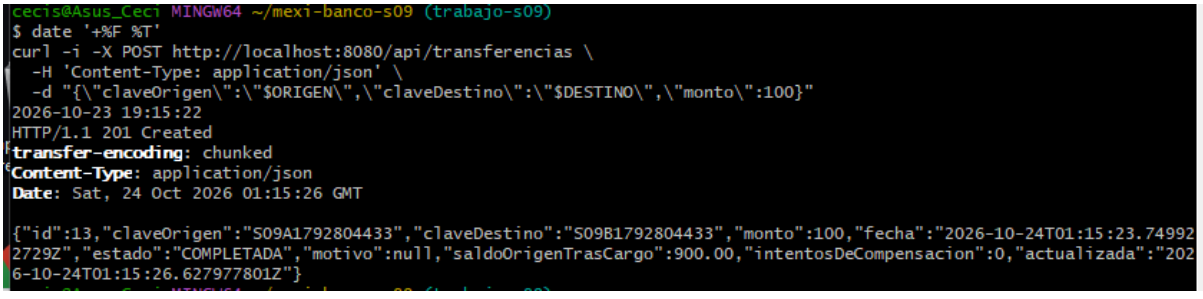
Preparación de las cuentas de prueba

El 23 de octubre de 2026, a las 19:13:56, inicié la creación de dos cuentas para S09 mediante el gateway. Origen quedó con CLABE S09A1792804433 y saldo 1000; destino, con CLABE S09B1792804433 y saldo 500. Ambas solicitudes respondieron 201 Created. El total inicial fue de 1500.



Comprobación de la versión base

El 23 de octubre de 2026, a las 19:15:22, solicité una transferencia de 100 con todos los servicios disponibles. La operación recibió el ID 13 y respondió 201 Created, con estado COMPLETADA. Esto confirmó que la versión base funcionaba antes de modificar el código.



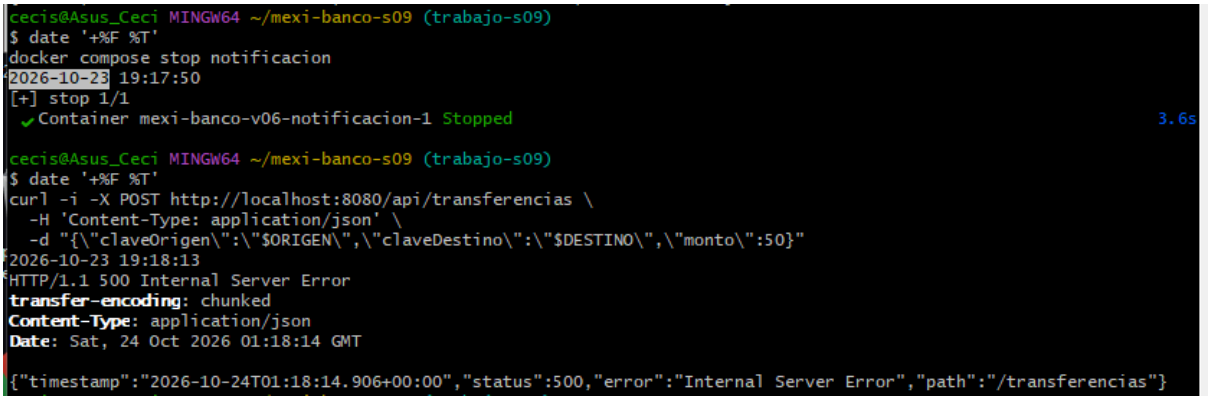
Saldos antes de la falla

El 23 de octubre de 2026, a las 19:16:33, consulté ambas cuentas. Origen tenía 900.00 y destino 600.00, con un total de 1500.00. Estos son los saldos previos a la prueba con notificación detenido.



Falla con notificación detenido

El 23 de octubre de 2026, a las 19:17:50, detuve notificación. A las 19:18:13 solicité una transferencia de 50. La solicitud respondió 500 Internal Server Error. No repetí la operación; consulté los saldos y el estado para comprobar qué pasos se habían completado.



Resultado de la falla

A las 19:19:14 consulté las cuentas y encontré saldos de 850.00 en origen y 650.00 en destino. Los 50 sí se transfirieron y el total permaneció en 1500.00.

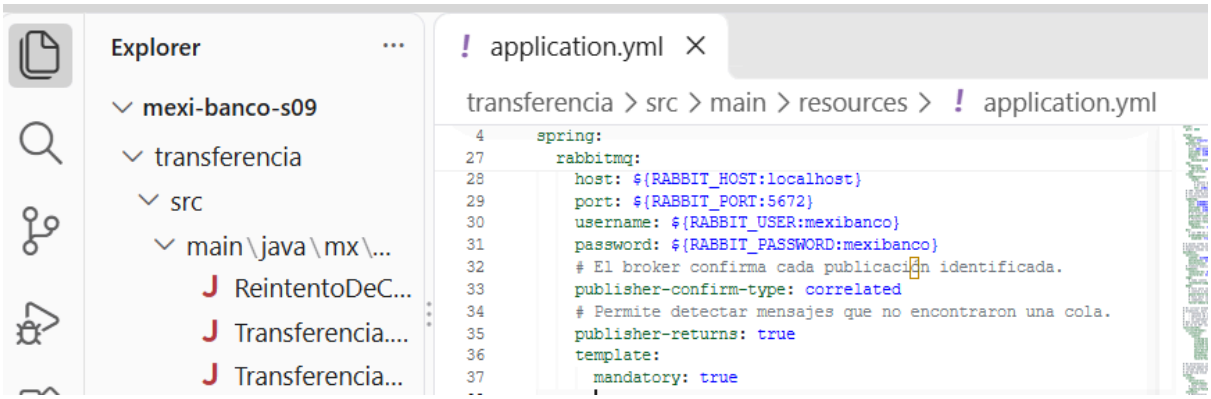
La operación 14 quedó en ABONADA. Aunque el dinero llegó al destino, la solicitud respondió con error porque todavía dependía de una llamada HTTP a notificación, que estaba detenido.



Configuración del publicador

Activé las confirmaciones correlacionadas de RabbitMQ para conocer si el broker confirmó una publicación. También habilité las devoluciones de mensajes sin ruta hacia una cola.

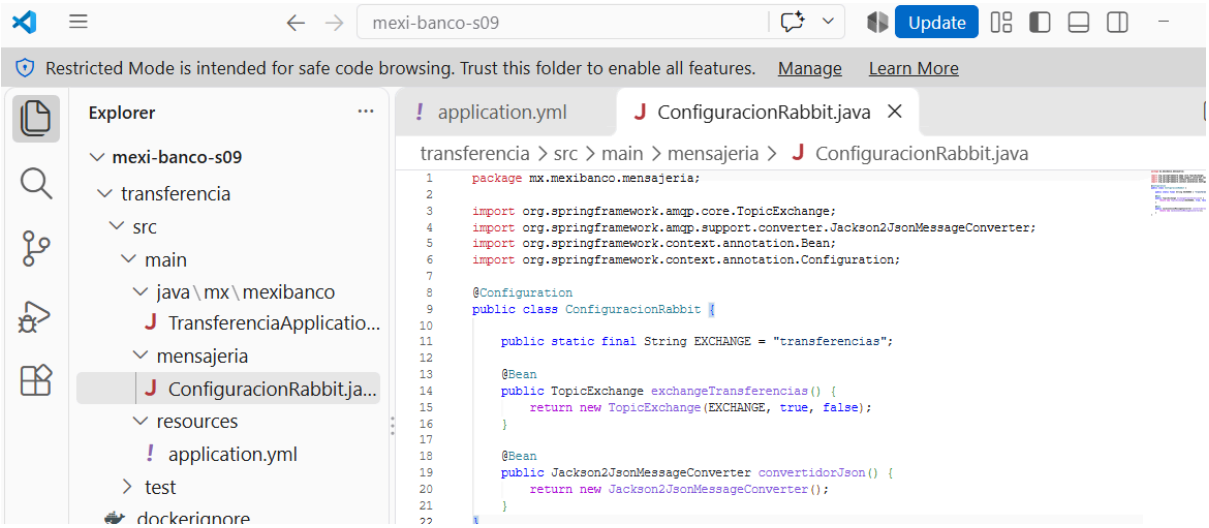
La confirmación del broker no demuestra que notificación haya leído o guardado el aviso. Un mensaje puede ser confirmado y, aun así, no tener una cola de destino. Esta configuración tampoco resuelve por sí sola la pérdida del evento si la base confirma la transferencia y el broker está caído.



Exchange y conversión a JSON

Creé ConfiguracionRabbit para declarar el exchange transferencias de tipo topic y durable. Este exchange permite enrutar eventos según su clave de publicación.

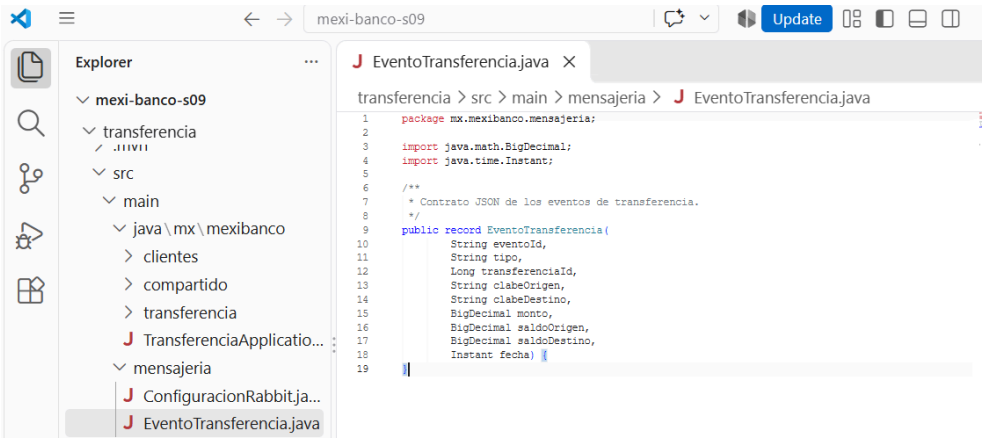
También declaré Jackson2JsonMessageConverter para que RabbitTemplate convierta los objetos de evento a JSON. El exchange por sí solo no almacena los mensajes para un consumidor: necesitamos una cola vinculada, que agregaremos en la Parte 2.



Contrato del evento

Creé EventoTransferencia con los nombres de campos indicados en la guía. El evento incluye su identificador, tipo, transferencia, cuentas, monto, saldos y fecha.

eventoId identificará el evento y deberá conservarse si vuelve a publicarse. El consumidor utilizará ese identificador para reconocer duplicados. Crear el record solo define el contrato; todavía falta generar y conservar el identificador y conectar la publicación.

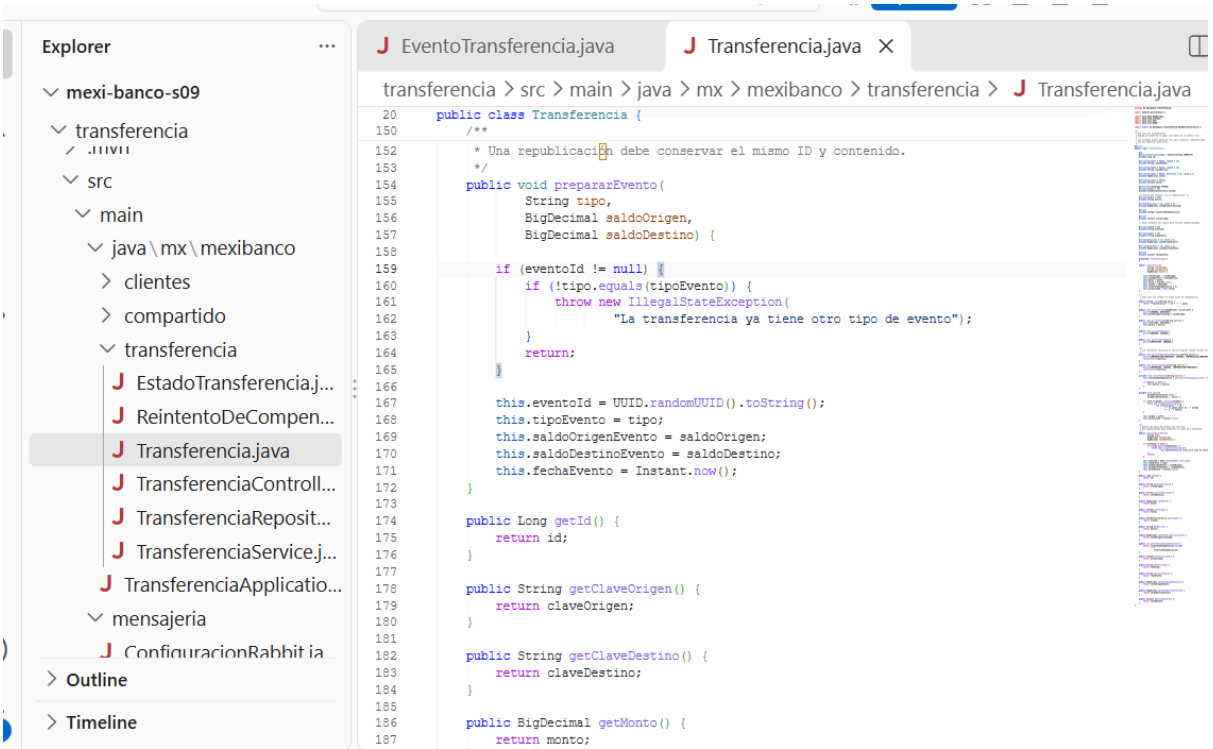


Identidad estable del evento

Agregué a la transferencia los datos del evento: identificador, tipo, saldos y fecha. El método prepararEvento genera un UUID únicamente la primera vez y conserva los valores si se vuelve a utilizar.

Estos datos se guardarán junto con el estado final antes de publicar. Esto permite conservar el mismo eventoId al republicar, pero todavía no es un Transactional Outbox: no hay un proceso que busque y publique automáticamente eventos pendientes.

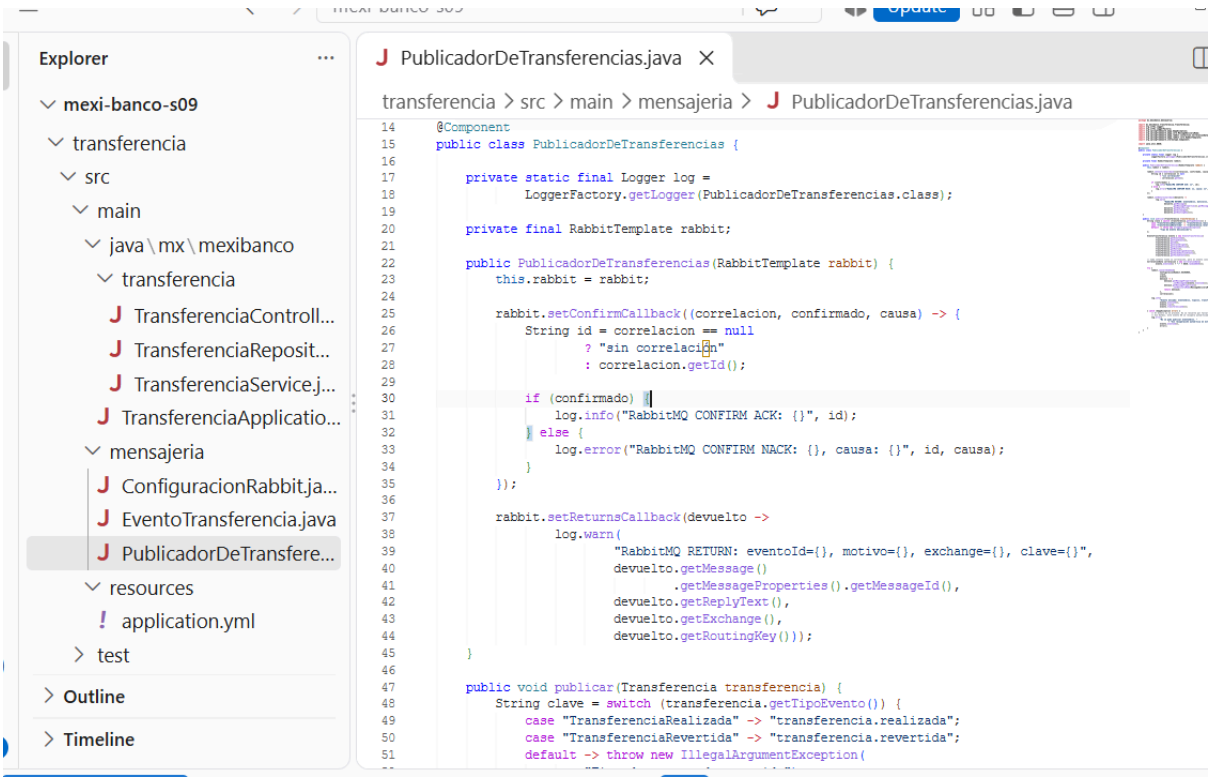
```
56
57      @Column(length = 36)
58      private String eventoId;
59
60      @Column(length = 40)
61      private String tipoEvento;
62
63      @Column(precision = 19, scale = 2)
64      private BigDecimal saldoOrigenEvento;
65
66      @Column(precision = 19, scale = 2)
67      private BigDecimal saldoDestinoEvento;
68
69      @Column
70      private Instant fechaEvento;
```

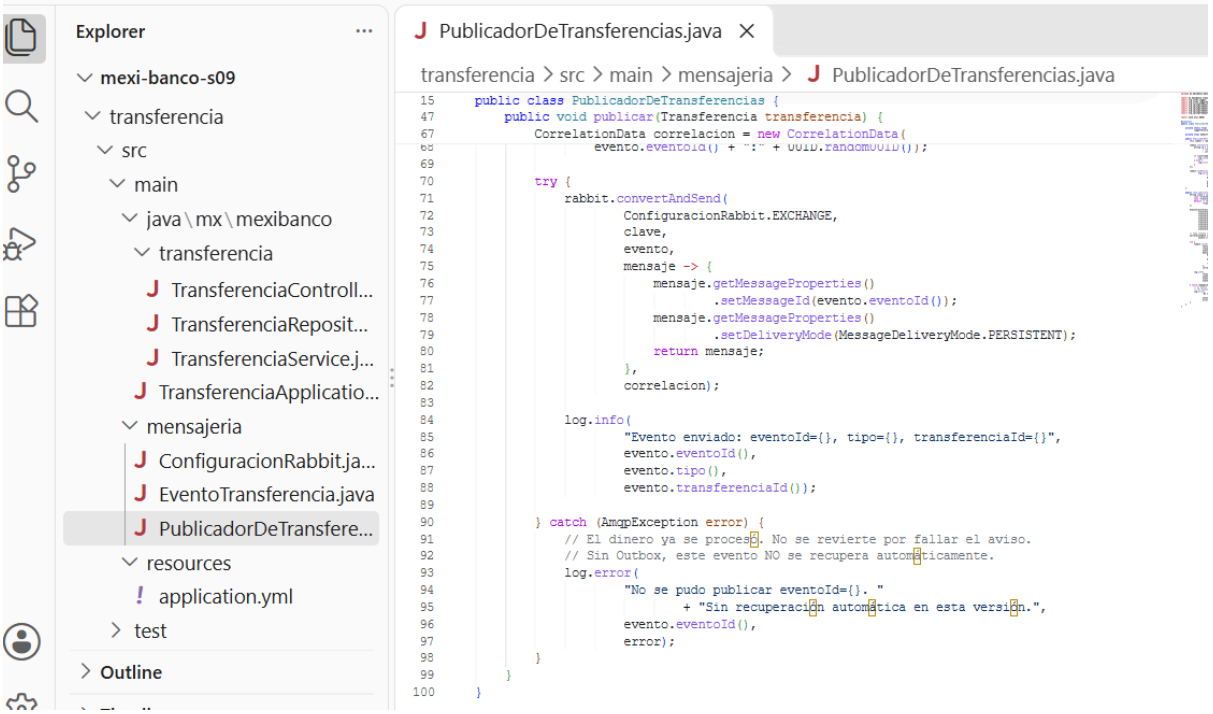


Publicador de eventos

Creé PublicadorDeTransferencias para enviar los eventos al exchange transferencias. Utiliza transferencia.realizada para una transferencia completada y transferencia.revertida para una compensada. El mensaje se marca como persistente y conserva su eventoId.

Los logs distinguen el envío, la confirmación del broker y la devolución por falta de una cola de destino. Si falla la publicación, se registra el error sin deshacer el movimiento de dinero. Esta versión todavía puede perder el aviso porque no tiene recuperación automática mediante Outbox.





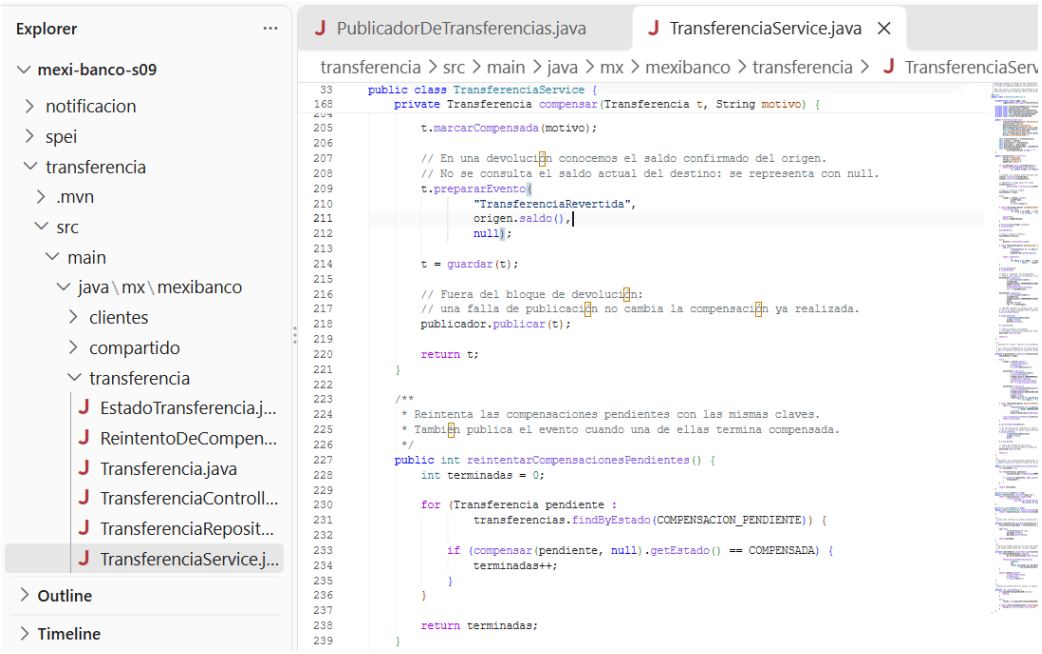
Publicación de una transferencia realizada

Reemplacé las dos llamadas HTTP a notificación por la publicación de TransferenciaRealizada. Después de registrar los movimientos, la saga se marca COMPLETADA y guarda los datos del evento. Luego lo publica con la clave transferencia.realizada. Así, transferencia ya no espera a que notificación genere los avisos.



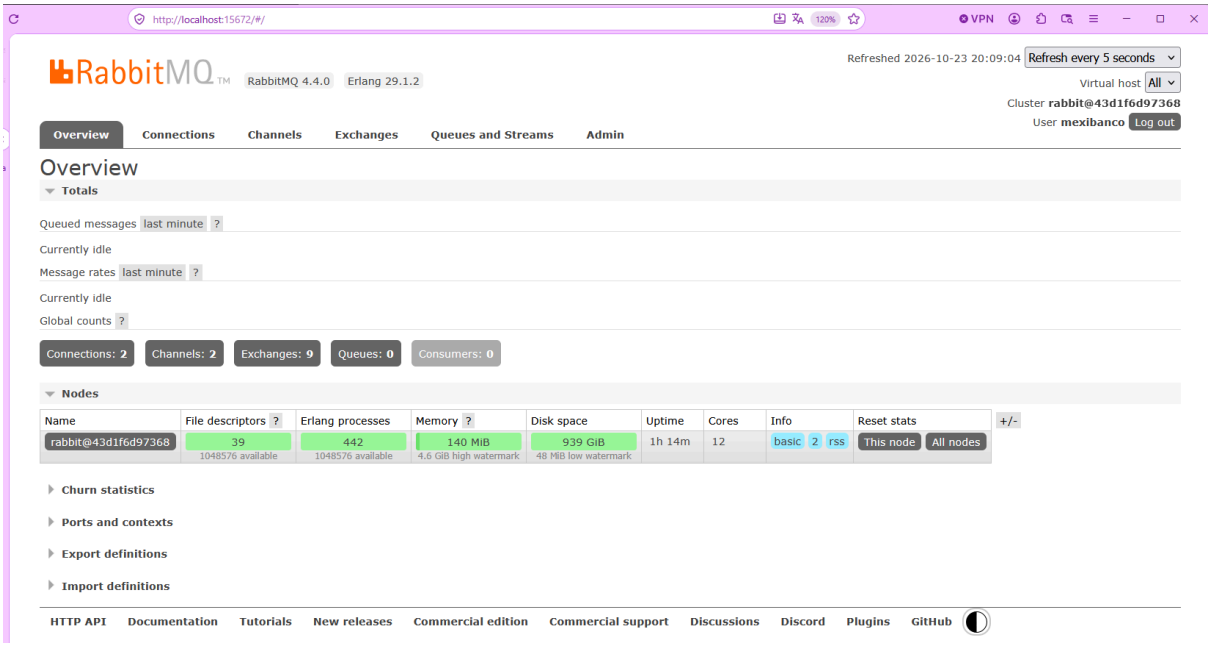
Publicación de una transferencia revertida

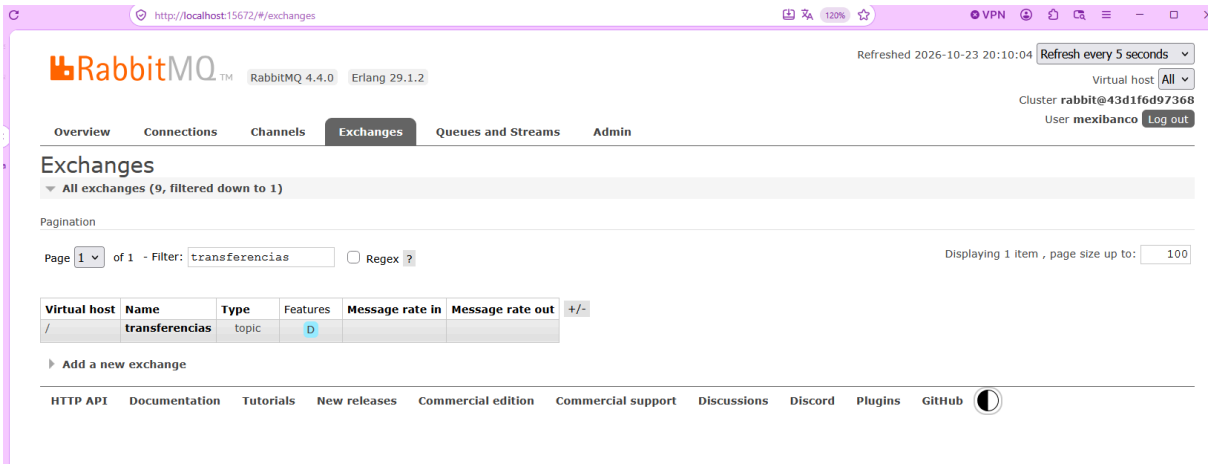
Cuando la devolución y sus movimientos se confirman, la saga se marca COMPENSADA, guarda los datos de TransferenciaRevertida y lo publica con la clave transferencia.revertida. Esto también ocurre si la compensación termina mediante el reintento programado. La publicación queda fuera del bloque de devolución para que una falla del broker no cambie una compensación ya realizada.



Comprobación del exchange

Accedí al panel de RabbitMQ y comprobé que todavía no había colas ni consumidores. También verifiqué que el exchange transferencias estaba declarado como topic y durable. En esta etapa, el exchange aún no tiene una cola vinculada que conserve los eventos para notificación.





Transferencia sin depender de notificación

El 23 de octubre de 2026 detuve notificación a las 20:11:15 y solicité una transferencia de 25 a las 20:11:28. La operación 15 respondió 201 Created y quedó COMPLETADA, aunque notificación seguía apagado.

La respuesta incluyó el evento TransferenciaRealizada con identificador 62cc58e3-160e-4fd2-850d-31da9b3c70bc y saldos de 825.00 y 675.00. Falta comprobar en los logs qué ocurrió al publicar, porque todavía no existe una cola de destino.



Confirmación sin cola de destino

Los logs muestran el envío del evento de la transferencia 15, una devolución NO_ROUTE y una confirmación ACK. RabbitMQ confirmó la publicación, pero no encontró una cola a la cual dirigir el mensaje.

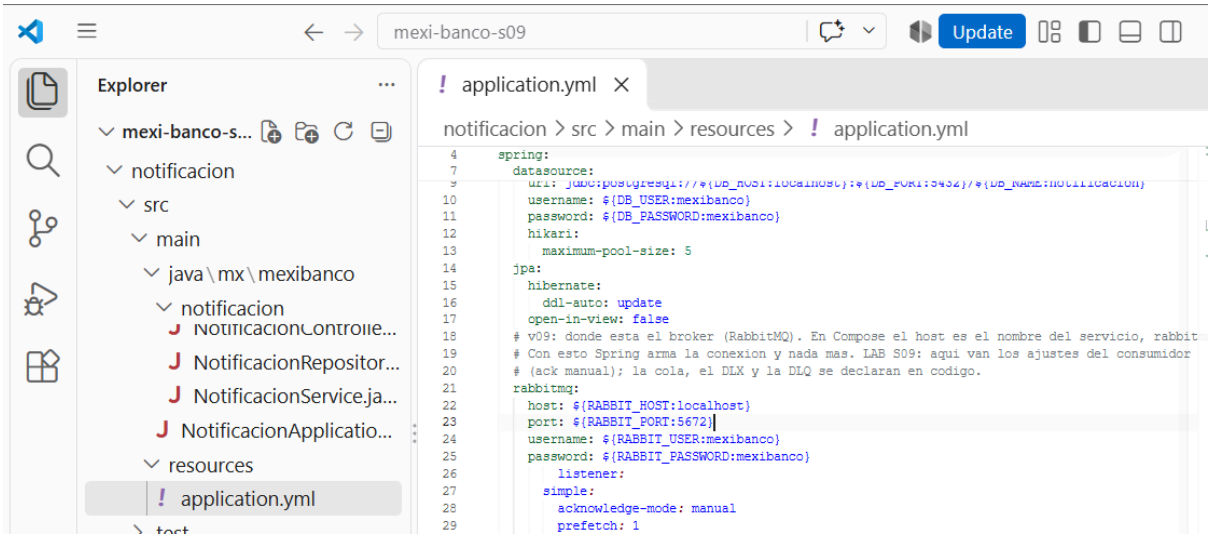
Como todavía no existe una cola vinculada y el publicador no almacena ni reintenta los mensajes devueltos, este evento no quedó pendiente para notificación. El publisher confirm no garantiza que un consumidor reciba o guarde el aviso.

```
cecis@Asus_Ceci MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ date '+%F %T'
curl -i -X POST http://localhost:8080/api/transferencias \
  -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"S09A1792804433","claveDestino":"S09B1792804433","monto":25}'
2026-10-23 20:11:28
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Sat, 24 Oct 2026 02:11:30 GMT

{"id":15,"claveOrigen":"S09A1792804433","claveDestino":"S09B1792804433","monto":25,"fecha":"2026-10-24T02:11:30.142411
146Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":825.00,"intentosDeCompensacion":0,"actualizada":"2026
-10-24T02:11:30.6954757542","eventoId":"62cc58e3-160e-4fd2-850d-31da9b3c70bc","tipoEvento":"TransferenciaRealizada","s
aldoOrigenEvento":825.00,"saldoDestinoEvento":675.00,"fechaEvento":"2026-10-24T02:11:30.696032935Z"}
cecis@Asus_Ceci MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose logs --since=10m transferencia | grep -E 'Evento enviado|RabbitMQ|No se pudo publicar'
transferencia-1  | 2026-10-24T02:11:30.799Z  INFO 1 --- [transferencia] [nio-8080-exec-2] m.m.m.PublicadorDeTransferen
cias         : Evento enviado: eventoId=62cc58e3-160e-4fd2-850d-31da9b3c70bc, tipo=TransferenciaRealizada, transferenc
iaId=15
transferencia-1  | 2026-10-24T02:11:30.824Z  WARN 1 --- [transferencia] [nectionFactory2] m.m.m.PublicadorDeTransferen
cias         : RabbitMQ RETURN: eventoId=62cc58e3-160e-4fd2-850d-31da9b3c70bc, motivo=NO_ROUTE, exchange=transferencia
s, clave=transferencia.realizada
transferencia-1  | 2026-10-24T02:11:30.837Z  INFO 1 --- [transferencia] [nectionFactory2] m.m.m.PublicadorDeTransferen
cias         : RabbitMQ CONFIRM ACK: 62cc58e3-160e-4fd2-850d-31da9b3c70bc:4197ef62-d159-4c66-aa45-e39ef44ee354
```

Configuración del consumidor

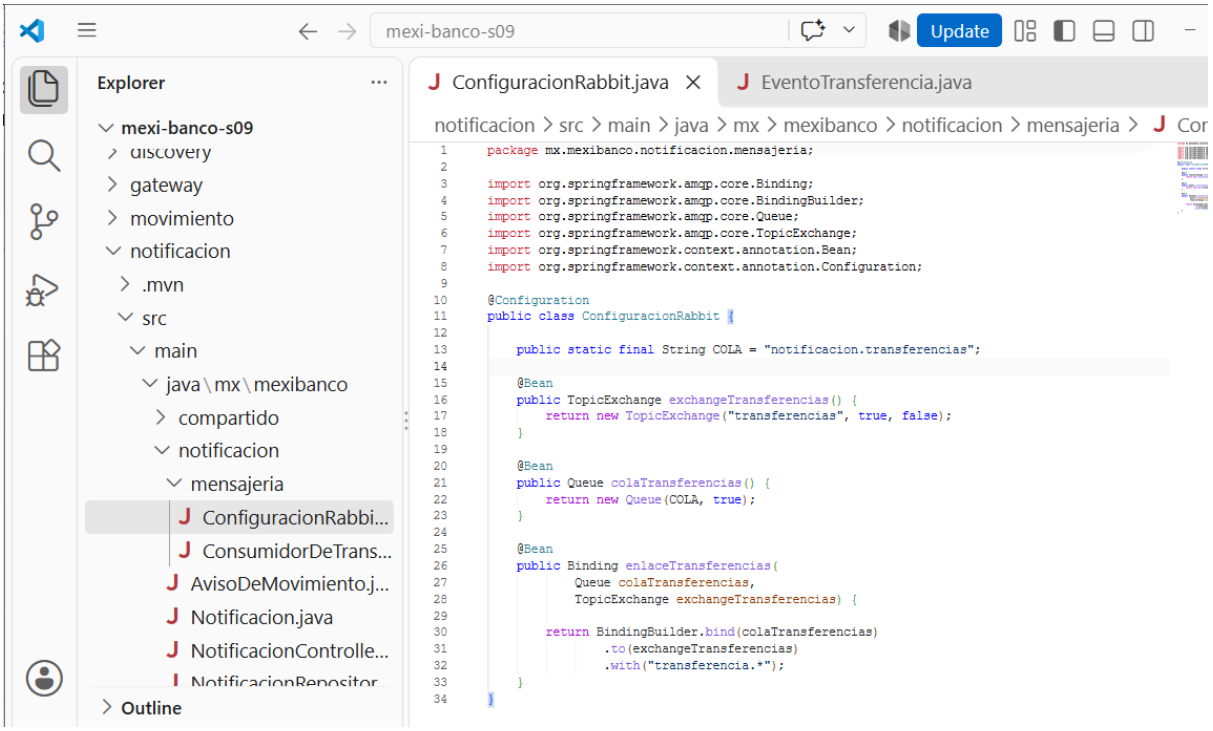
Configuré notificación con confirmación manual y prefetch de 1. El consumidor deberá guardar los avisos antes de enviar el ack. Si pierde la conexión antes de confirmarlo, RabbitMQ podrá volver a entregar el mensaje. Esta configuración todavía no evita avisos duplicados.



Cola de notificación

Declaré la cola durable notificacion.transferencias y la vinculé al exchange transferencias con el patrón transferencia.*. Esto permitirá recibir los eventos publicados con las claves transferencia.realizada y transferencia.revertida.

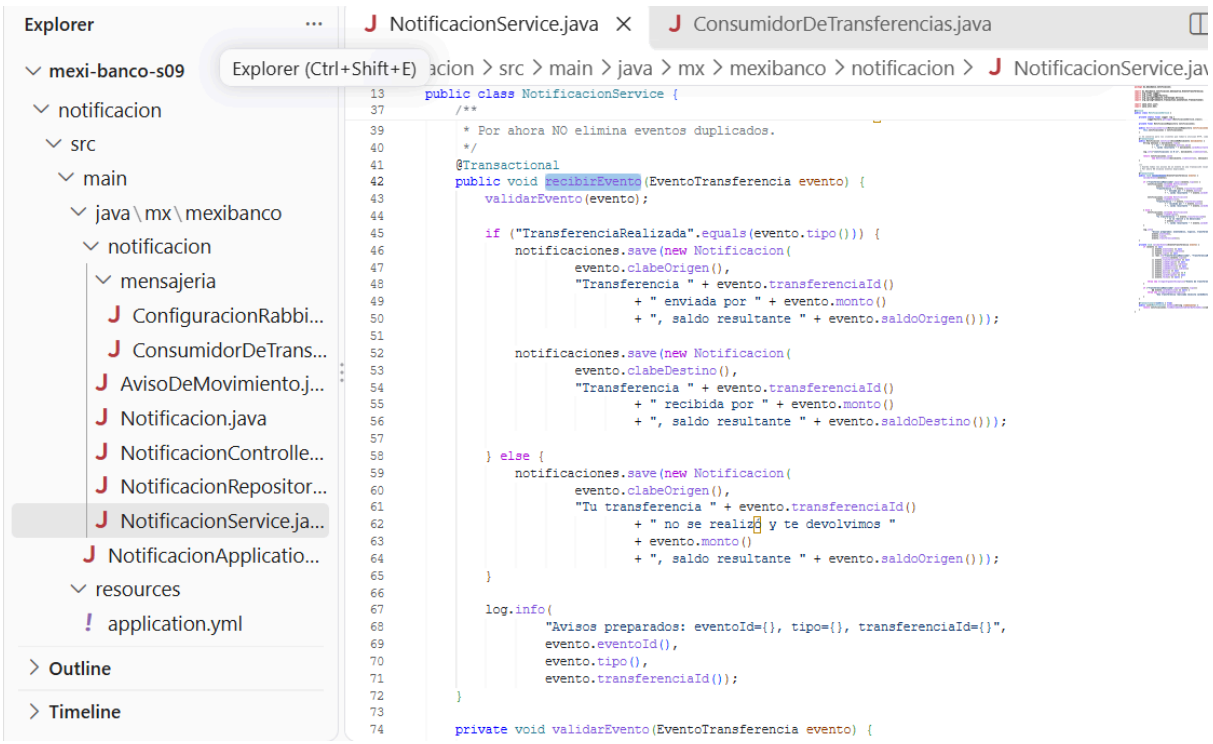
En esta etapa todavía no agregué deduplicación ni DLQ, porque primero comprobaré el consumidor básico y reproduciré esos problemas.

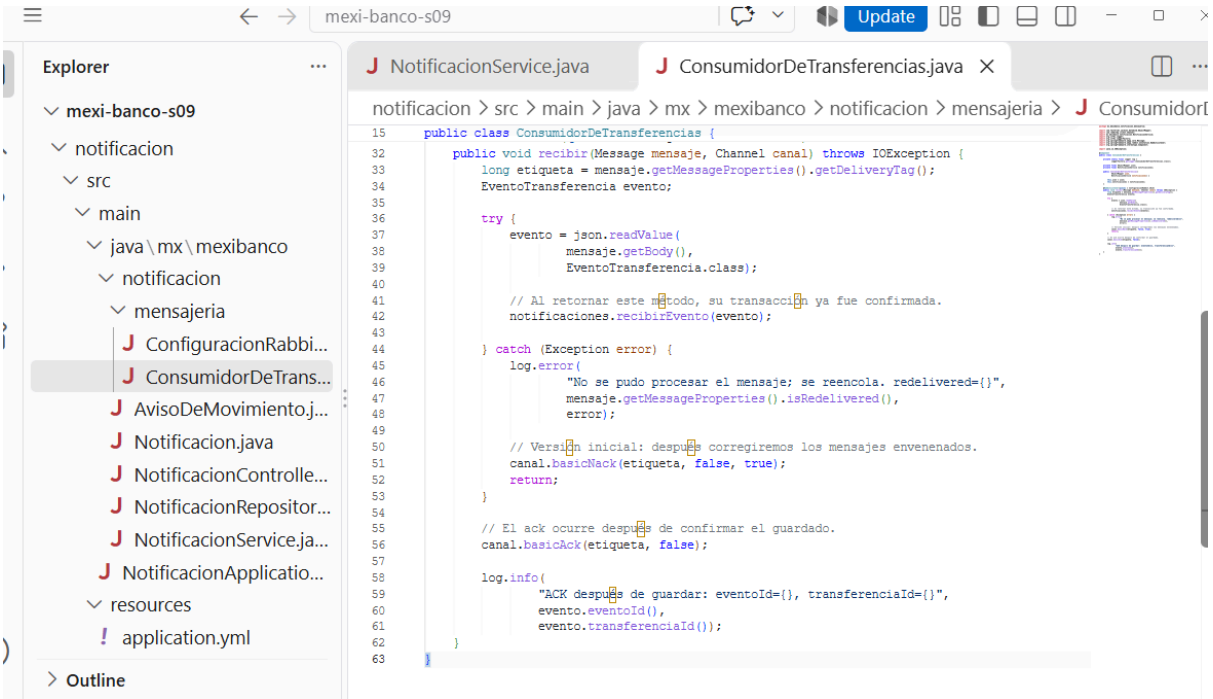


Guardado y confirmación del evento

Implementé el consumidor para leer el JSON y guardar dos avisos por TransferenciaRealizada, uno para cada cuenta, y uno al origen por TransferenciaRevertida.

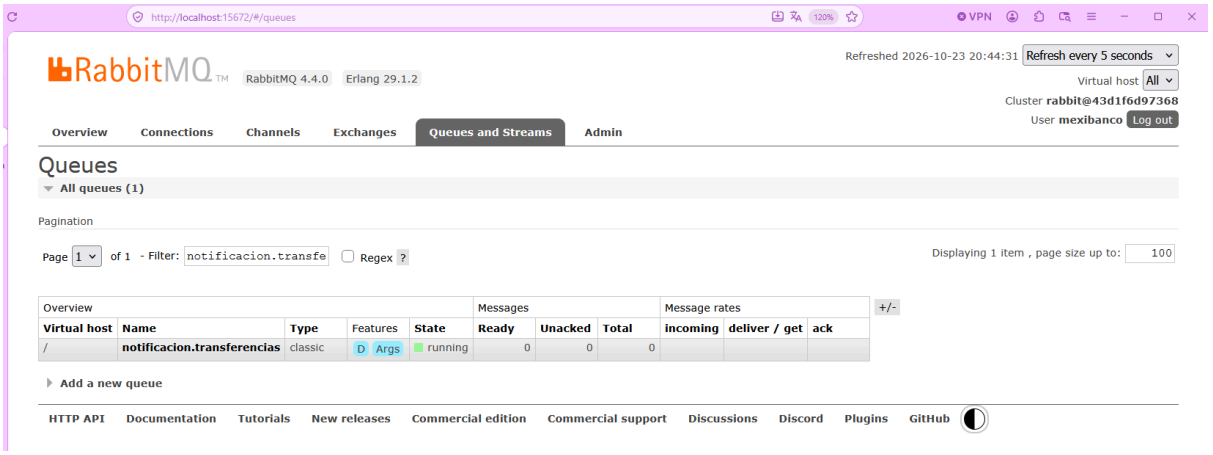
Los avisos de cada evento se guardan en una sola transacción. El consumidor envía el ack únicamente después de que esa transacción se confirma. Si el guardado falla, esta primera versión reencola el mensaje. Todavía permite avisos duplicados y no tiene tratamiento definitivo para mensajes inválidos.

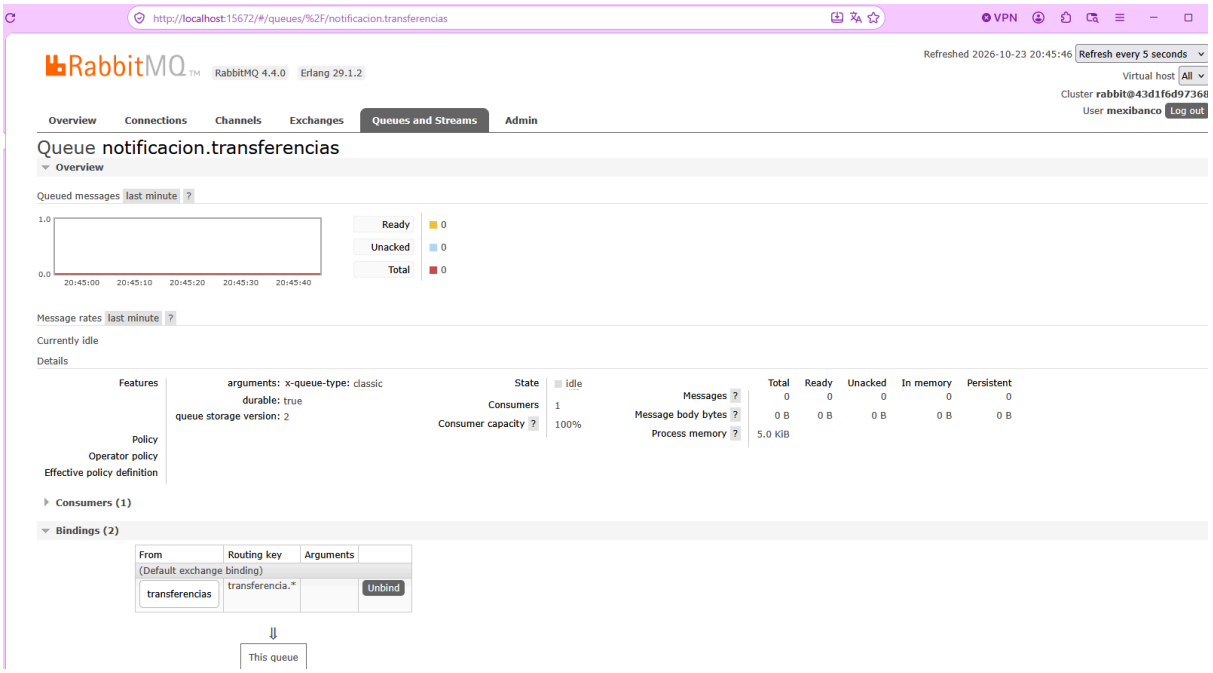

Explorer
mexi-banco-s09
Explorer (Ctrl+Shift+E)
notificacion
src
main
java \ mx \ mexibanco
notificacion
mensajeria
ConfiguracionRabbi...
ConsumidorDeTrans...
AvisoDeMovimiento.j...
Notificacion.java
NotificacionControlle...
NotificacionRepositor...
NotificacionService.ja...
NotificacionApplicatio...
resources
application.yml
Outline
Timeline
NotificacionService.java
ConsumidorDeTransferencias.java
acion > src > main > java > mx > mexibanco > notificacion > NotificacionService.jav
13 public class NotificacionService {
37 /**
39 * Por ahora NO elimina eventos duplicados.
40 */
41 @Transactional
42 public void recibirEvento(EventoTransferencia evento) {
43 validarEvento(evento);
44
45 if ("TransferenciaRealizada".equals(evento.tipo())) {
46 notificaciones.save(new Notificacion(
47 evento.clabeOrigen(),
48 "Transferencia " + evento.transferenciaId()
49 + " enviada por " + evento.monto()
50 + ", saldo resultante " + evento.saldoOrigen()));
51
52 notificaciones.save(new Notificacion(
53 evento.clabeDestino(),
54 "Transferencia " + evento.transferenciaId()
55 + " recibida por " + evento.monto()
56 + ", saldo resultante " + evento.saldoDestino()));
57
58 } else {
59 notificaciones.save(new Notificacion(
60 evento.clabeOrigen(),
61 "Tu transferencia " + evento.transferenciaId()
62 + " no se realizó y te devolvimos "
63 + evento.monto()
64 + ", saldo resultante " + evento.saldoOrigen()));
65 }
66
67 log.info(
68 "Avisos preparados: eventoId={}, tipo={}, transferenciaId={}",
69 evento.eventoId(),
70 evento.tipo(),
71 evento.transferenciaId());
72 }
73
74 private void validarEvento(EventoTransferencia evento) {


mexi-banco-s09
Update
Explorer
mexi-banco-s09
notificacion
src
main
java \ mx \ mexibanco
notificacion
mensajeria
ConfiguracionRabbi...
ConsumidorDeTrans...
AvisoDeMovimiento.j...
Notificacion.java
NotificacionControlle...
NotificacionRepositor...
NotificacionService.ja...
NotificacionApplicatio...
resources
application.yml
Outline
NotificacionService.java
ConsumidorDeTransferencias.java
notificacion > src > main > java > mx > mexibanco > notificacion > mensajeria > ConsumidorT
15 public class ConsumidorDeTransferencias {
32 public void recibir(Message mensaje, Channel canal) throws IOException {
33 long etiqueta = mensaje.getMessageProperties().getDeliveryTag();
34 EventoTransferencia evento;
35
36 try {
37 evento = json.readValue(
38 mensaje.getBody(),
39 EventoTransferencia.class);
40
41 // Al retornar este método, su transacción ya fue confirmada.
42 notificaciones.recibirEvento(evento);
43
44 } catch (Exception error) {
45 log.error(
46 "No se pudo procesar el mensaje; se reencola. redelivered={}",
47 mensaje.getMessageProperties().isRedelivered(),
48 error);
49
50 // Versión inicial: después corregiremos los mensajes envenenados.
51 canal.basicNack(etiqueta, false, true);
52 return;
53 }
54
55 // El ack ocurre después de confirmar el guardado.
56 canal.basicAck(etiqueta, false);
57
58 log.info(
59 "ACK después de guardar: eventoId={}, transferenciaId={}",
60 evento.eventoId(),
61 evento.transferenciaId());
62 }
63 }

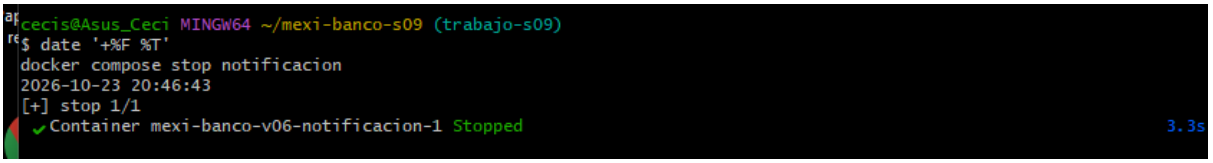
Declaración de la cola

El 23 de octubre de 2026, aproximadamente a las 20:45, comprobé que la cola notificacion.transferencias estaba creada como durable y tenía un consumidor conectado. Sus contadores estaban en cero. También verifiqué su enlace al exchange transferencias mediante el patrón transferencia.*.



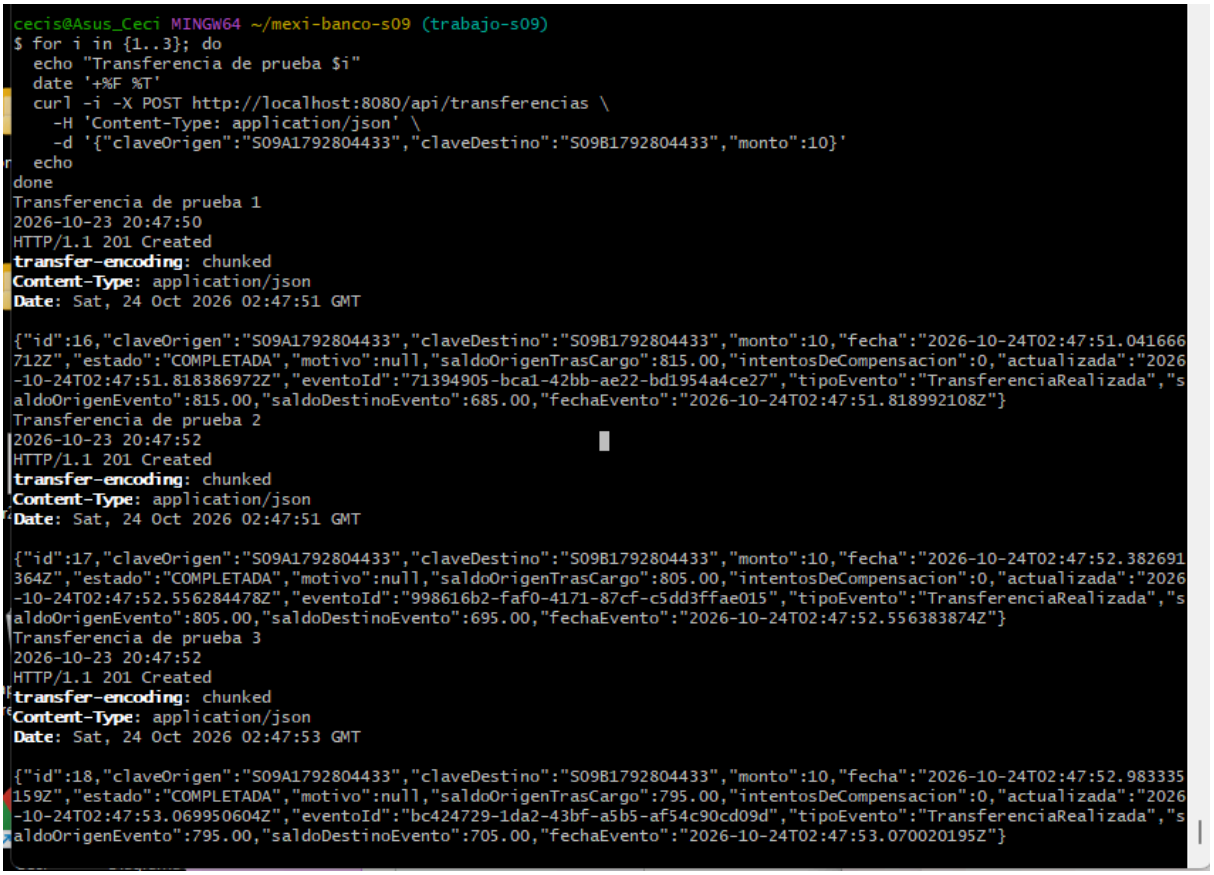


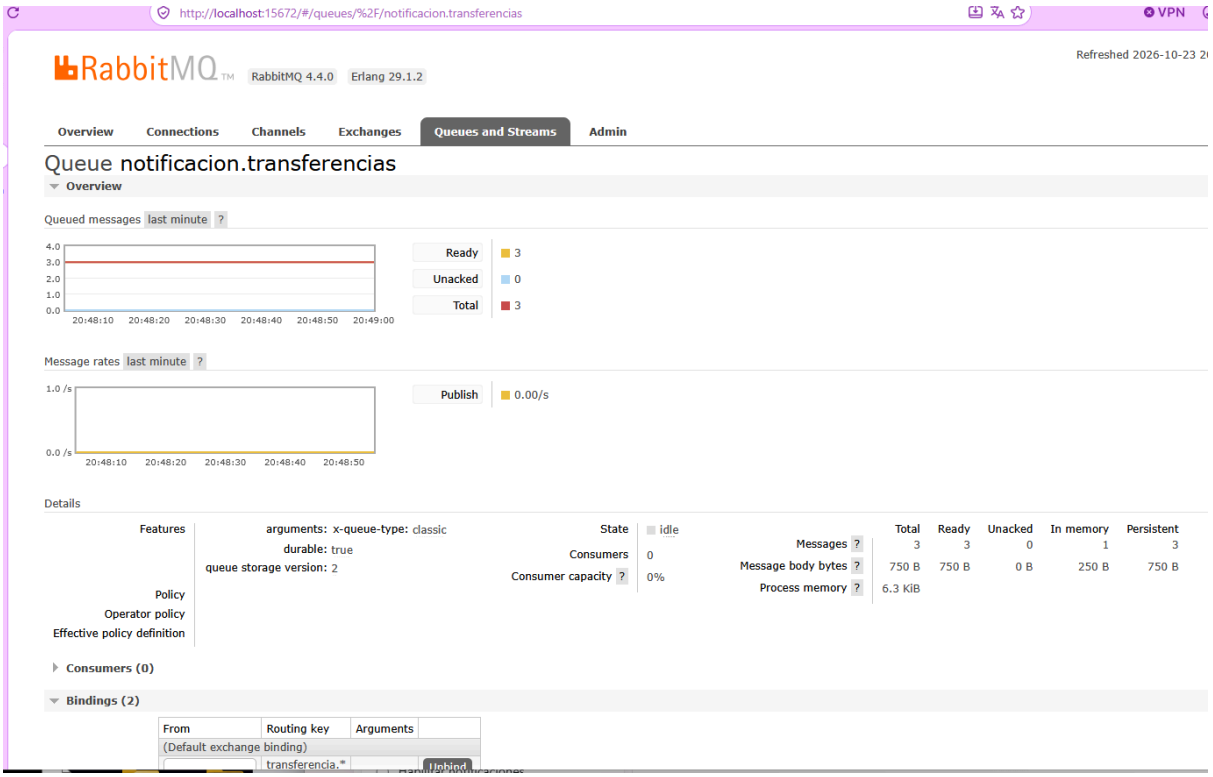
Notificación quedó detenido a las 20:46:43.



Eventos en espera

Con notificación detenido, la cola notificacion.transferencias conservó tres mensajes Ready, ninguno sin confirmar y ningún consumidor conectado. Los eventos quedaron esperando en RabbitMQ aunque las transferencias ya habían terminado.





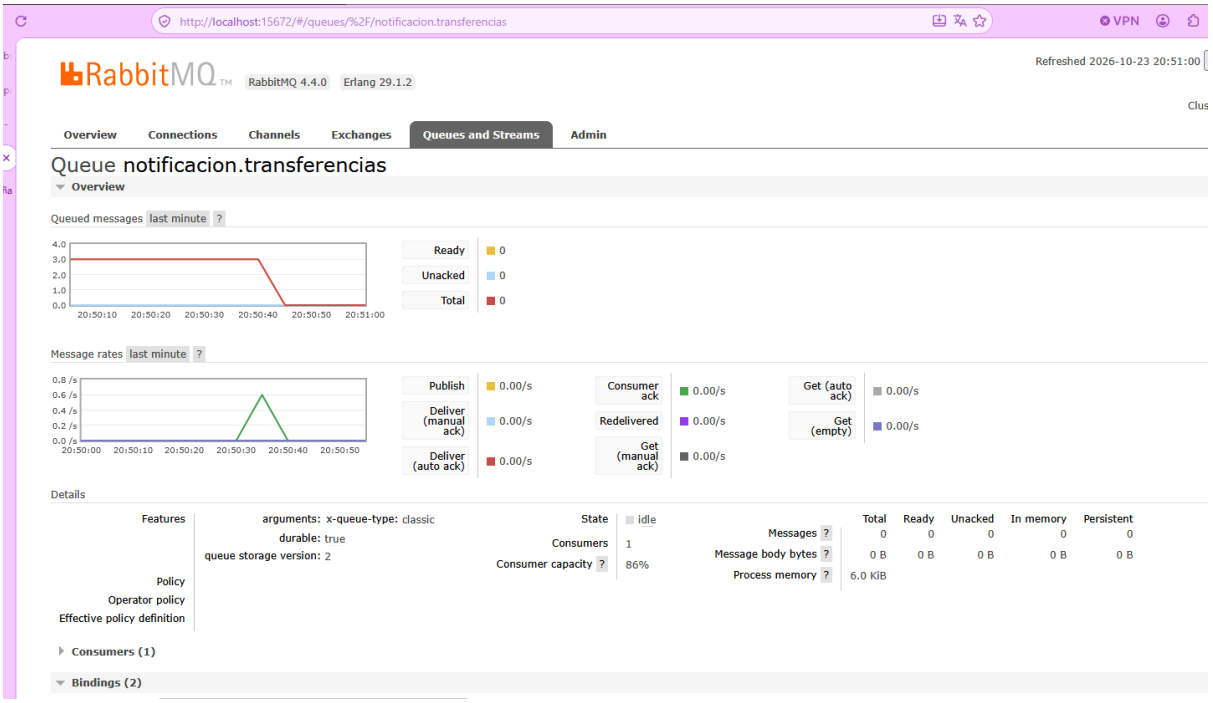
Procesamiento después de recuperar el consumidor

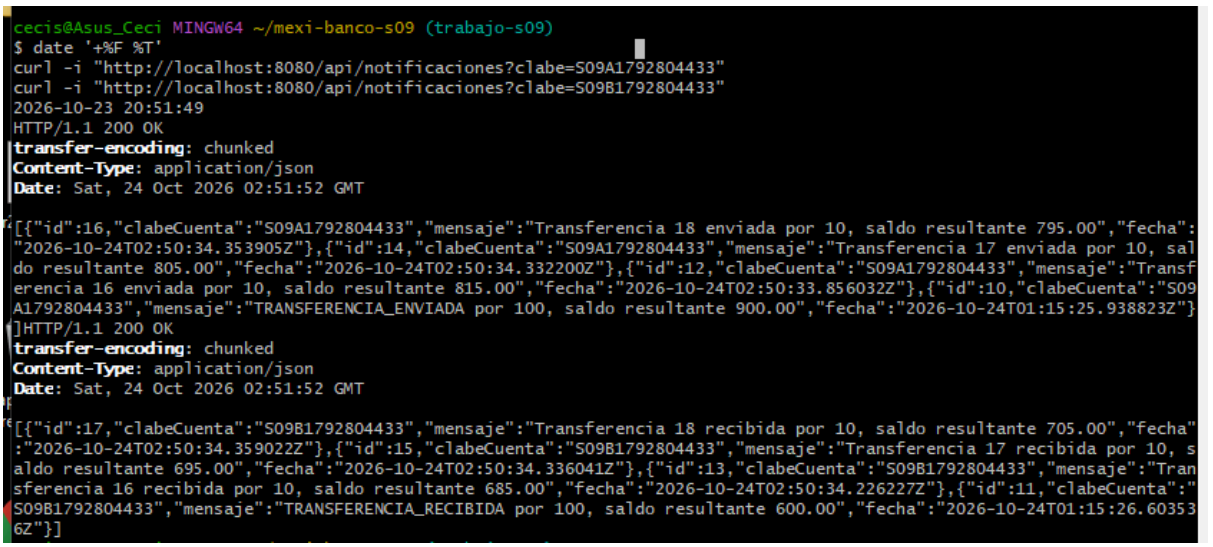
A las 20:51:49 consulté las bandejas y encontré los avisos de las transferencias 16, 17 y 18 en ambas cuentas: tres de envío en origen y tres de recepción en destino.

Esto demuestra el desacople en tiempo: las transferencias terminaron mientras notificación estaba detenido, RabbitMQ conservó los eventos y el consumidor guardó los avisos después de recuperarse.

```
cecis@Asus_Ceci MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ date '+%F %T'
docker compose start notificacion
2026-10-23 20:50:03
[+] start 4/4
 ✔ Container mexi-banco-v06-discovery-1    Healthy                                0.5s
 ✔ Container mexi-banco-v06-rabbitmq-1     Healthy                                0.5s
 ✔ Container mexi-banco-v06-db-1           Healthy                                0.5s
 ✔ Container mexi-banco-v06-notificacion-1 Started                                0.9s

cecis@Asus_Ceci MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose ps notificacion
NAME                          IMAGE                          COMMAND              SERVICE        CREATED         STA
TUS                      PORTS
mexi-banco-v06-notificacion-1   mexi-banco-v06-notificacion   "java -jar app.jar"   notificacion   9 minutes ago   Up
33 seconds (healthy)   8080/tcp
```





Preparación del evento de reversión

El 23 de octubre de 2026, a las 20:54:44, comprobé que FALLAR_ABONO_A_CLABE apuntaba a la cuenta destino y que la pausa era de 0 milisegundos. Los saldos iniciales eran 795.00 en origen y 705.00 en destino. Mantuve cuenta y notificación disponibles para comprobar la compensación y su aviso.

```
cecis@Asus_Ceci MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ FALLAR_ABONO_A_CLABE=S09B1792804433 docker compose up -d --no-deps transferencia
[+] up 1/1
 ✔ Container mexi-banco-v06-transferencia-1 Started                                4.6s
```

```
cecis@Asus_Ceci MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose ps transferencia
NAME                              IMAGE                          COMMAND              SERVICE         CREATED
      STATUS                        PORTS
mexi-banco-v06-transferencia-1    mexi-banco-v06-transferencia   "java -jar app.jar"   transferencia   About a minute a
go    Up About a minute (healthy)   8080/tcp

cecis@Asus_Ceci MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ date '+%F %T'
docker compose exec -T transferencia printenv FALLAR_ABONO_A_CLABE PAUSA_ENTRE_CARGO_Y_ABONO_MS
curl -i "http://localhost:8080/api/cuentas/S09A1792804433"
curl -i "http://localhost:8080/api/cuentas/S09B1792804433"
2026-10-23 20:54:44
S09B1792804433
0
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: b86f28af5d93
Content-Type: application/json
Date: Sat, 24 Oct 2026 02:54:47 GMT

{"id":6,"clabe":"S09A1792804433","titular":"Origen S09","saldo":795.00}HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 0060a8ef0aa5
Content-Type: application/json
Date: Sat, 24 Oct 2026 02:54:48 GMT

{"id":7,"clabe":"S09B1792804433","titular":"Destino S09","saldo":705.00}
```

Primer resultado de la prueba de reversión

El 23 de octubre de 2026, a las 20:56:07, solicité una transferencia de 20 con la falla del abono activa. La operación 19 respondió 502 y quedó en COMPENSACION_PENDIENTE. Revisé los logs y los saldos antes de concluir si la devolución se había aplicado, porque este estado también puede deberse a una falla al registrar sus movimientos.

```
cecis@Asus_Ceci MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ date '+%F %T'
curl -i -X POST http://localhost:8080/api/transferencias \
  -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"S09A1792804433","claveDestino":"S09B1792804433","monto":20}'
2026-10-23 20:56:07
HTTP/1.1 502 Bad Gateway
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Sat, 24 Oct 2026 02:56:10 GMT

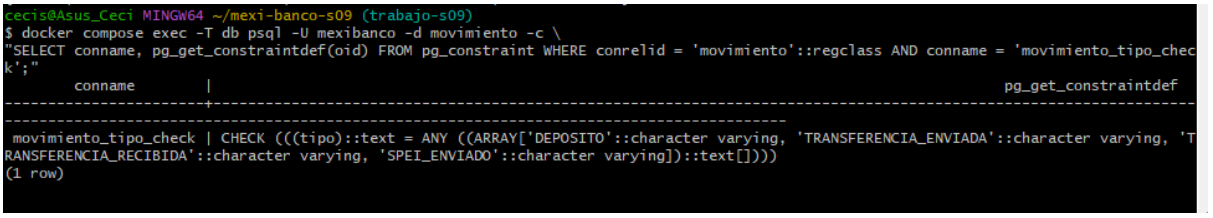
{"type":"about:blank","title":"Transferencia no realizada","status":502,"detail":"No se pudo abonar a la CLABE S09B179
2804433 y la devolucion de 20 a la CLABE S09A1792804433 tampoco pudo hacerse todavia. El dinero salio de origen; el ba
nco reintentara la devolucion automaticamente.","instance":"/transferencias","transferencia":19,"estado":"COMPENSACION
_PENDIENTE","motivo":"El abono a la CLABE S09B1792804433 fallo: Falla provocada con FALLAR_ABONO_A_CLABE=S09B179280443
3"}
```

Revisión de la compensación

Al consultar las cuentas, origen tenía nuevamente 795.00 y destino conservaba 705.00. Esto confirmó que los 20 sí se devolvieron, aunque la transferencia seguía en COMPENSACION_PENDIENTE.

Revisé los logs y encontré que PostgreSQL rechazaba el movimiento DEVOLUCION_TRANSFERENCIA por la restricción movimiento_tipo_check. La restricción conservaba los cuatro tipos de una versión anterior, mientras que el código actual ya incluía el tipo de devolución.





Corrección de la restricción

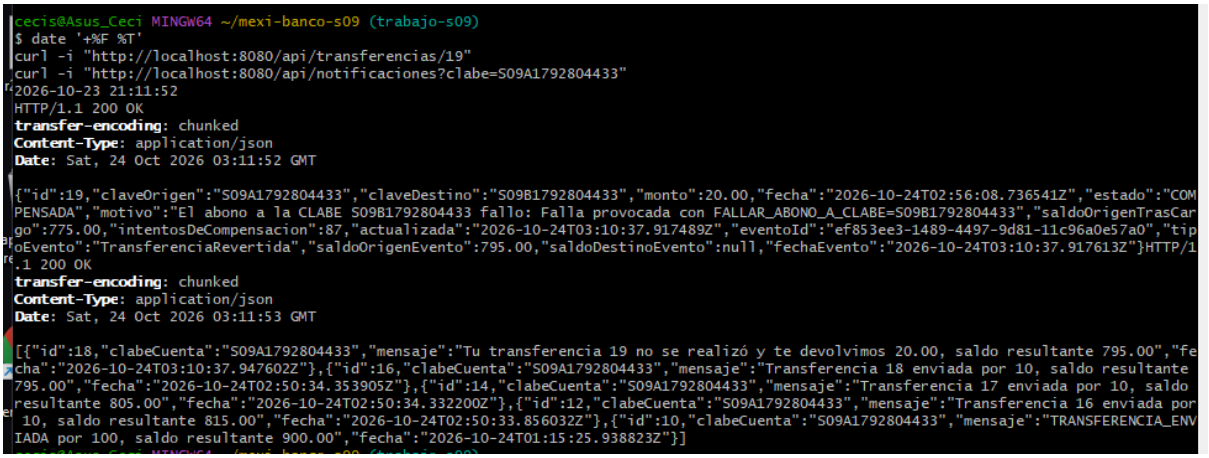
Actualicé movimiento_tipo_check para permitir DEVOLUCION_TRANSFERENCIA, conservando los cuatro tipos anteriores. El cambio terminó en COMMIT y no eliminó registros. Después dejé que el reintento automático continuara la compensación de la transferencia 19.

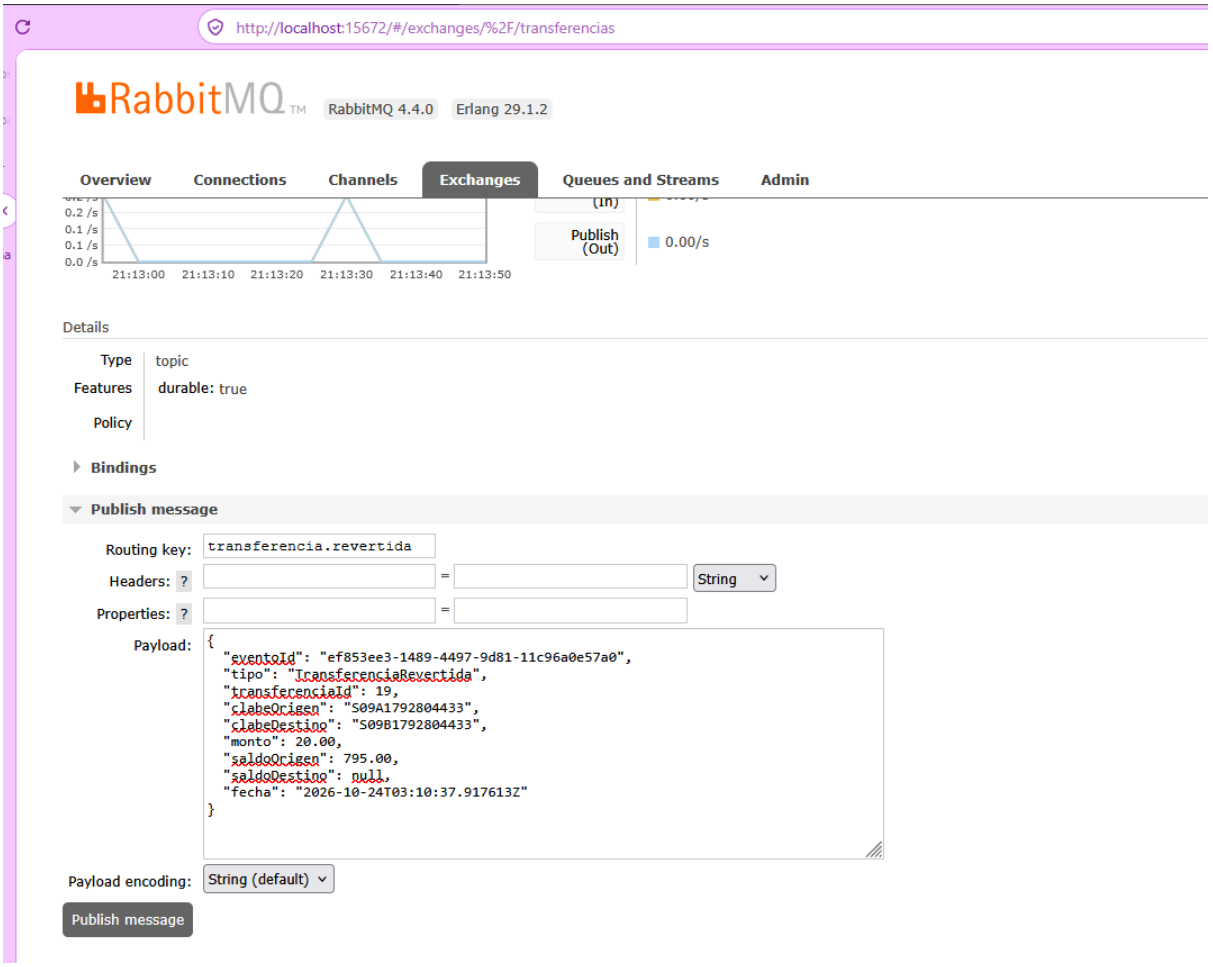
```
cecis@Asus_Ceci MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose exec -T db psql -v ON_ERROR_STOP=1 -U mexibanco -d movimiento -c "
BEGIN;
ALTER TABLE movimiento DROP CONSTRAINT movimiento_tipo_check;
ALTER TABLE movimiento ADD CONSTRAINT movimiento_tipo_check
CHECK (tipo IN (
  'DEPOSITO',
  'TRANSFERENCIA_ENVIADA',
  'TRANSFERENCIA_RECIBIDA',
  'SPEI_ENVIADO',
  'DEVOLUCION_TRANSFERENCIA'
));
COMMIT;
"
BEGIN
ALTER TABLE
ALTER TABLE
COMMIT
```

Reversión y aviso confirmados

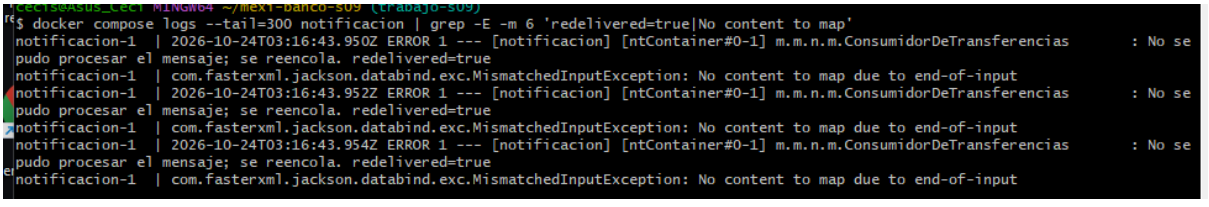
A las 21:11:52 consulté la transferencia 19 y comprobé que había pasado a COMPENSADA. El reintento automático pudo finalizar después de corregir la restricción de movimientos.

La operación generó el evento TransferenciaRevertida con identificador ef853ee3-1489-4497-9d81-11c96a0e57a0. En la bandeja de origen apareció el aviso de que la transferencia no se realizó y se devolvieron 20.00, con saldo resultante 795.00.



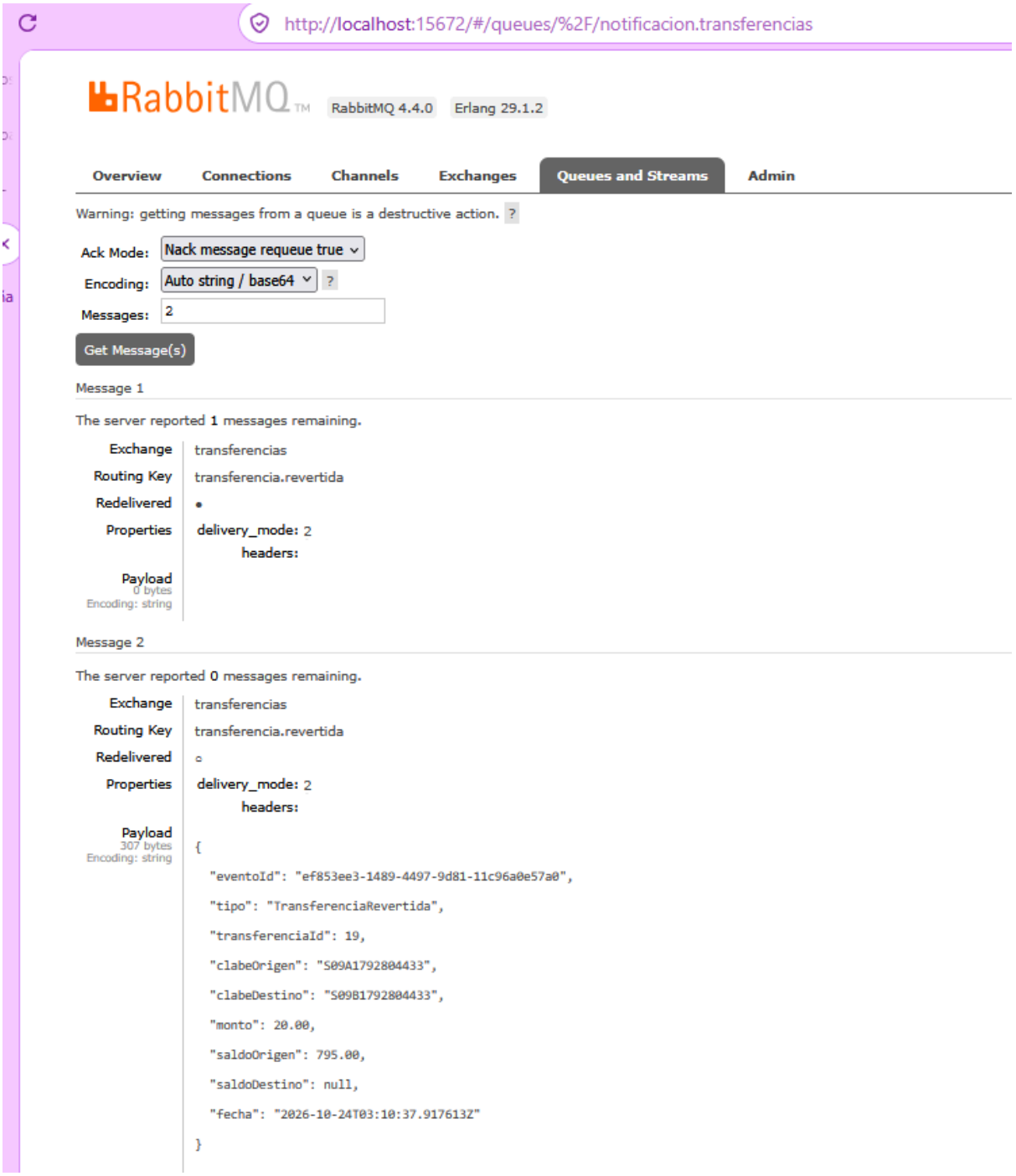


Durante la prueba de republicación, el consumidor recibió un mensaje sin contenido y no pudo convertirlo a JSON. Como estaba configurado para reencolar los errores, el mensaje se entregó repetidamente. Detuve notificación para interrumpir el bucle. Esta prueba todavía no demuestra avisos duplicados.



Inspección de los mensajes

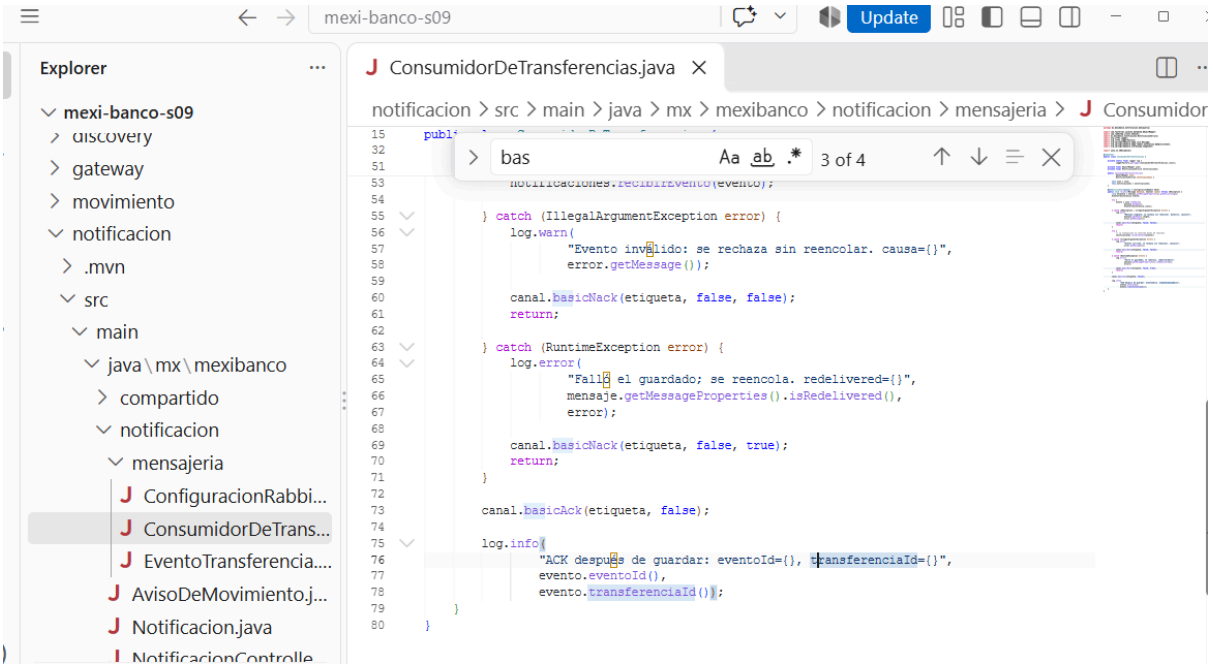
Con notificación detenido, inspeccioné los dos mensajes usando Nack message requeue true para conservarlos. El primero tenía un cuerpo vacío de 0 bytes; el segundo contenía el evento TransferenciaRevertida con el mismo eventoId de la transferencia 19. Esto explica por qué el consumidor registraba errores de lectura de JSON.



Tratamiento de mensajes inválidos

Modifiqué el consumidor para rechazar sin reencolar los mensajes que no se pueden leer como JSON o no cumplen la validación del evento. Configuré temporalmente el enlace al dead letter exchange mediante una política, conservando los mensajes de la cola existente.

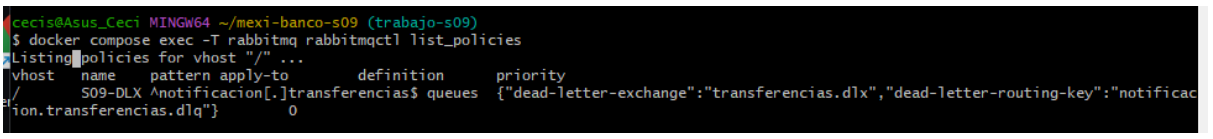
Los errores de guardado siguen reencolándose. La deduplicación todavía está pendiente.



Comprobación de la política de mensajes rechazados

Consulté las políticas de RabbitMQ y confirmé que S09-DLX se aplica únicamente a la cola notificacion.transferencias. Configura transferencias.dlx como dead letter exchange y notificacion.transferencias.dlq como clave de ruteo para los mensajes rechazados sin reencolar.

Esta comprobación muestra la configuración; todavía falta verificar que el mensaje inválido llegue a la DLQ.



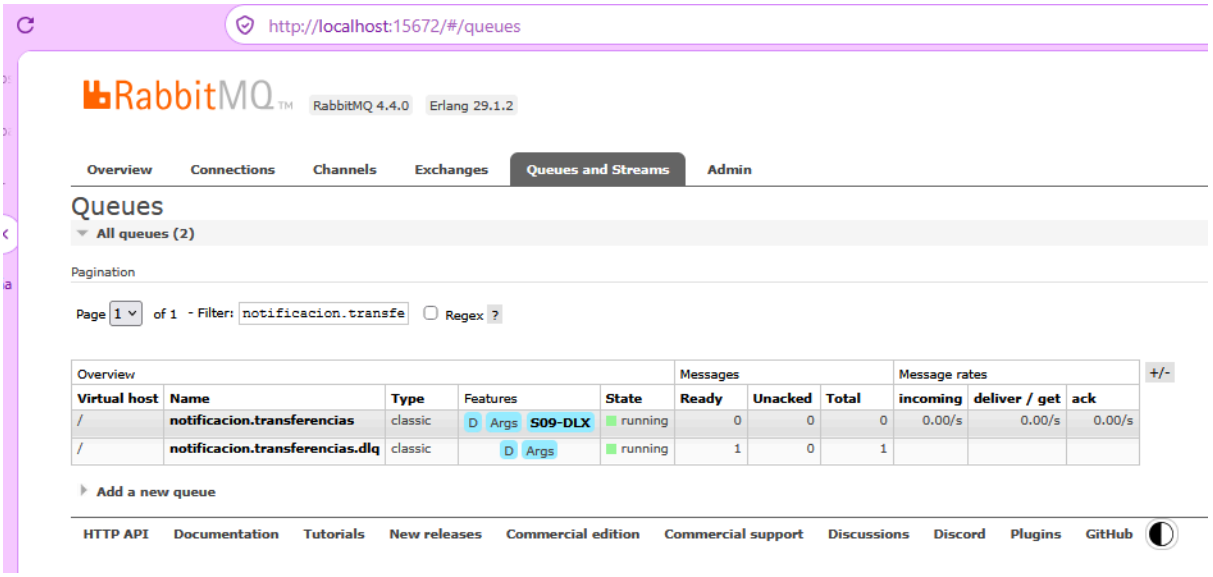
Mensaje aislado en la DLQ

Después de iniciar el consumidor corregido, la cola principal quedó sin mensajes y notificacion.transferencias.dlq mostró un mensaje Ready. El mensaje ilegible dejó de reencolarse en la cola principal. A continuación inspeccioné su contenido para confirmar que correspondía al cuerpo vacío.

```
cecis@Asus_Ceci MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ date '+%F %T'
docker compose up -d --no-deps notificacion
2026-10-23 21:36:08
[+] up 1/1
 ✔Container mexi-banco-v06-notificacion-1 Started                                                                    1.8s

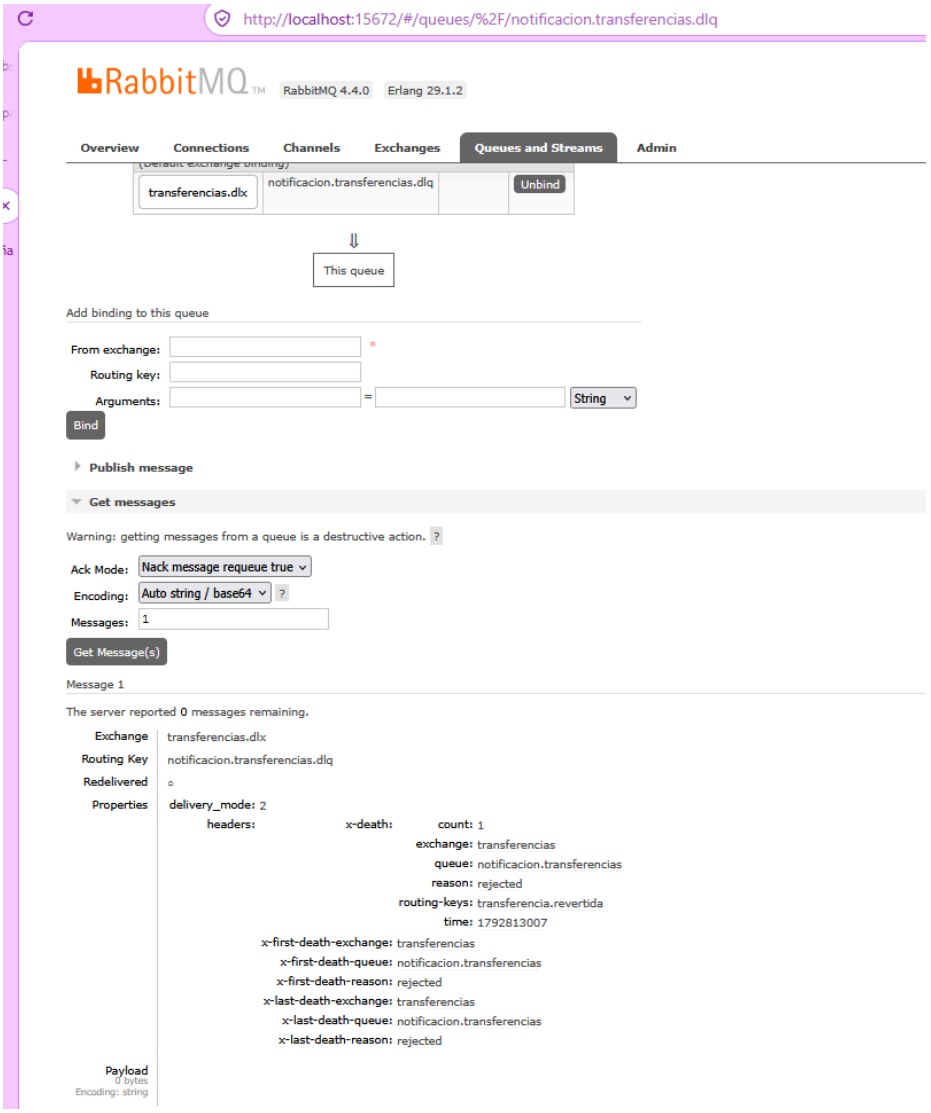
cecis@Asus_Ceci MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose ps notificacion
NAME                          IMAGE                         COMMAND              SERVICE        CREATED          STATUS
          PORTS
mexi-banco-v06-notificacion-1   mexi-banco-v06-notificacion   "java -jar app.jar"   notificacion   25 seconds ago   Up 24 seconds (health: starting)   80
80/tcp

cecis@Asus_Ceci MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose ps notificacion
NAME                          IMAGE                         COMMAND              SERVICE        CREATED          STATUS                    PORTS
mexi-banco-v06-notificacion-1   mexi-banco-v06-notificacion   "java -jar app.jar"   notificacion   42 seconds ago   Up 40 seconds (healthy)   8080/tcp
```



Inspección del mensaje rechazado

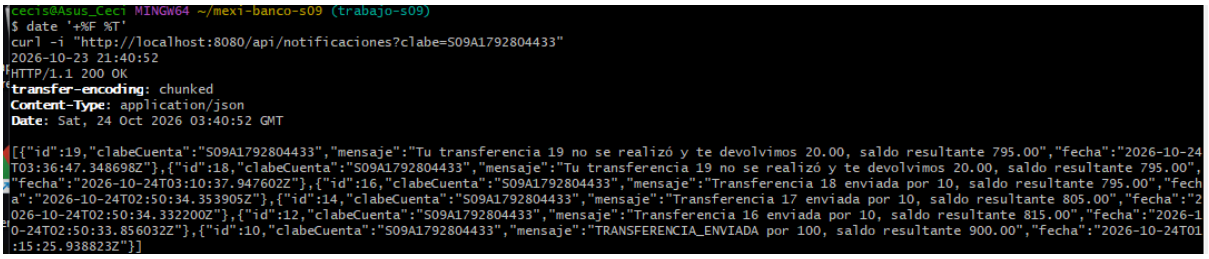
Inspeccioné la DLQ con Nack message requeue true y confirmé que el mensaje tenía un cuerpo vacío de 0 bytes. El encabezado x-death indica que fue rechazado desde notificacion.transferencias. El mensaje quedó separado de la cola principal para poder revisarlo sin mantener el bucle de reentregas.



Aviso duplicado

El 23 de octubre de 2026, a las 21:40:52, consulté la bandeja de origen. Encontré dos avisos de la misma transferencia 19, con identificadores de notificación 18 y 19.

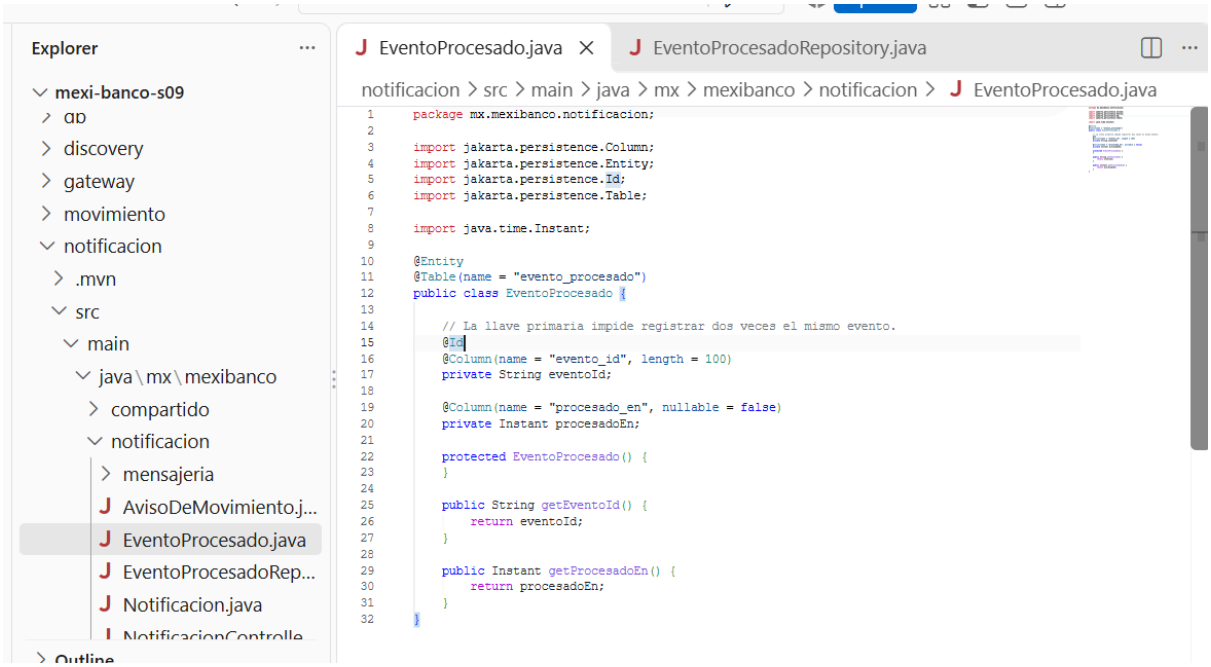
Al republicar el evento con el mismo eventoId, el consumidor volvió a guardar el aviso porque todavía no verificaba si ya lo había procesado. La republicación duplicó la notificación; no ejecutó otra transferencia ni otra devolución de dinero.

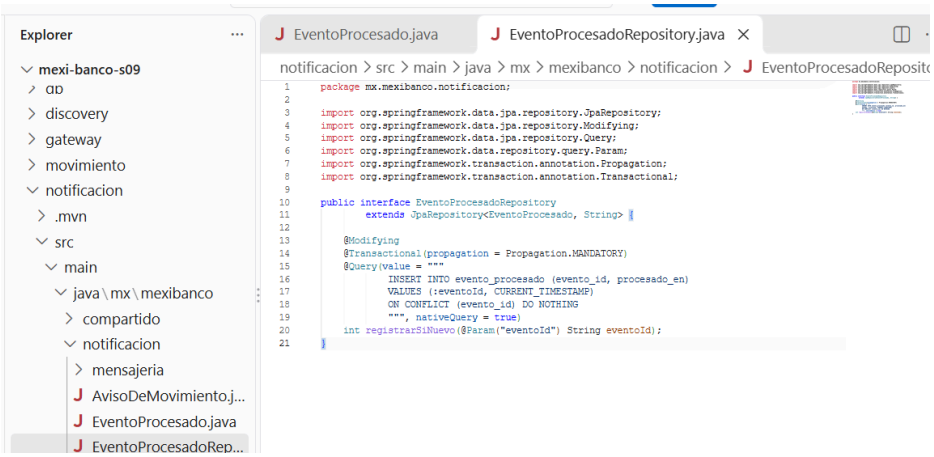


Registro de eventos procesados

Creé la tabla evento_procesado, donde eventoId es la llave primaria y no puede repetirse. La inserción utiliza ON CONFLICT DO NOTHING para reconocer un identificador existente, incluso si llegan dos entregas al mismo tiempo.

El registro del evento se integrará en la misma transacción que los avisos. No basta con consultar primero si existe, porque dos consumidores podrían hacer esa consulta simultáneamente y ambos intentar guardar.

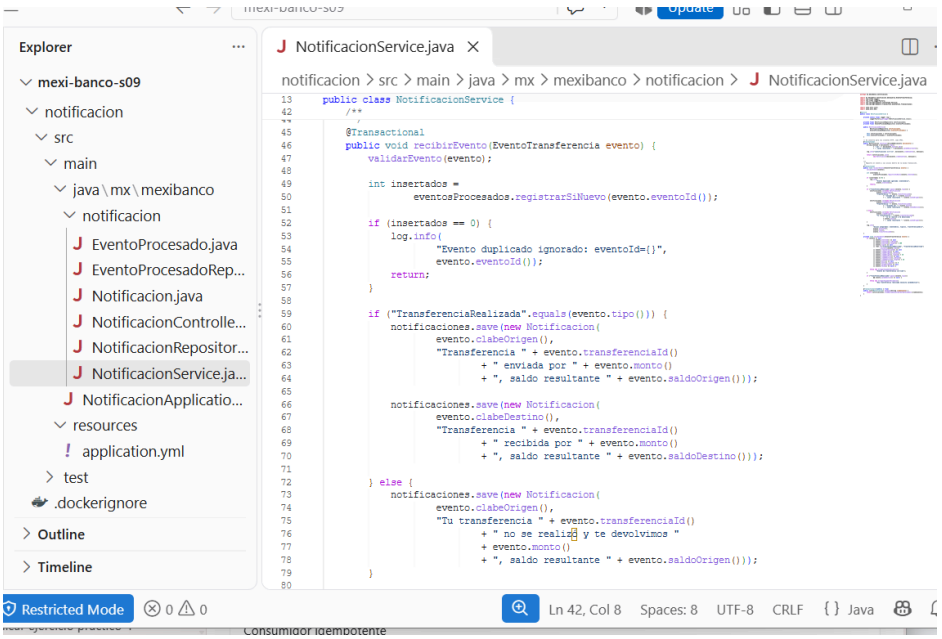




Consumidor idempotente

Integré el registro de eventoId y la creación de avisos en una misma transacción. Si el identificador ya existe, el método termina sin generar nuevos avisos y el consumidor confirma el mensaje.

Si ocurre un error al guardar los avisos, la transacción también revierte la inserción del identificador. Así, una entrega posterior puede volver a procesar el evento sin que se marque como terminado antes de guardar.



Preparación de la deduplicación

Inicié la versión de notificación con deduplicación y comprobé que la tabla evento_procesado existía y estaba vacía. Los avisos anteriores se conservaron; esta tabla comenzará a registrar los eventos recibidos por la versión nueva.



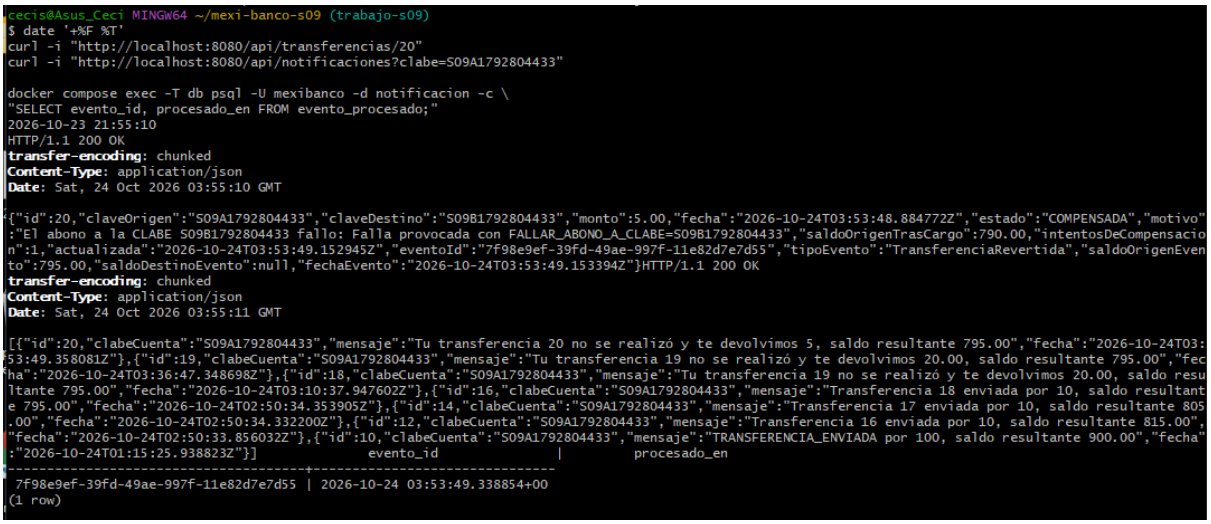
Evento nuevo para comprobar la deduplicación

El 23 de octubre de 2026, a las 21:53:47, solicité una transferencia de 5 con la falla del abono activa. La operación 20 quedó COMPENSADA y la respuesta indicó que el dinero fue devuelto. Utilicé esta operación para generar un evento nuevo, posterior a la incorporación de la deduplicación.

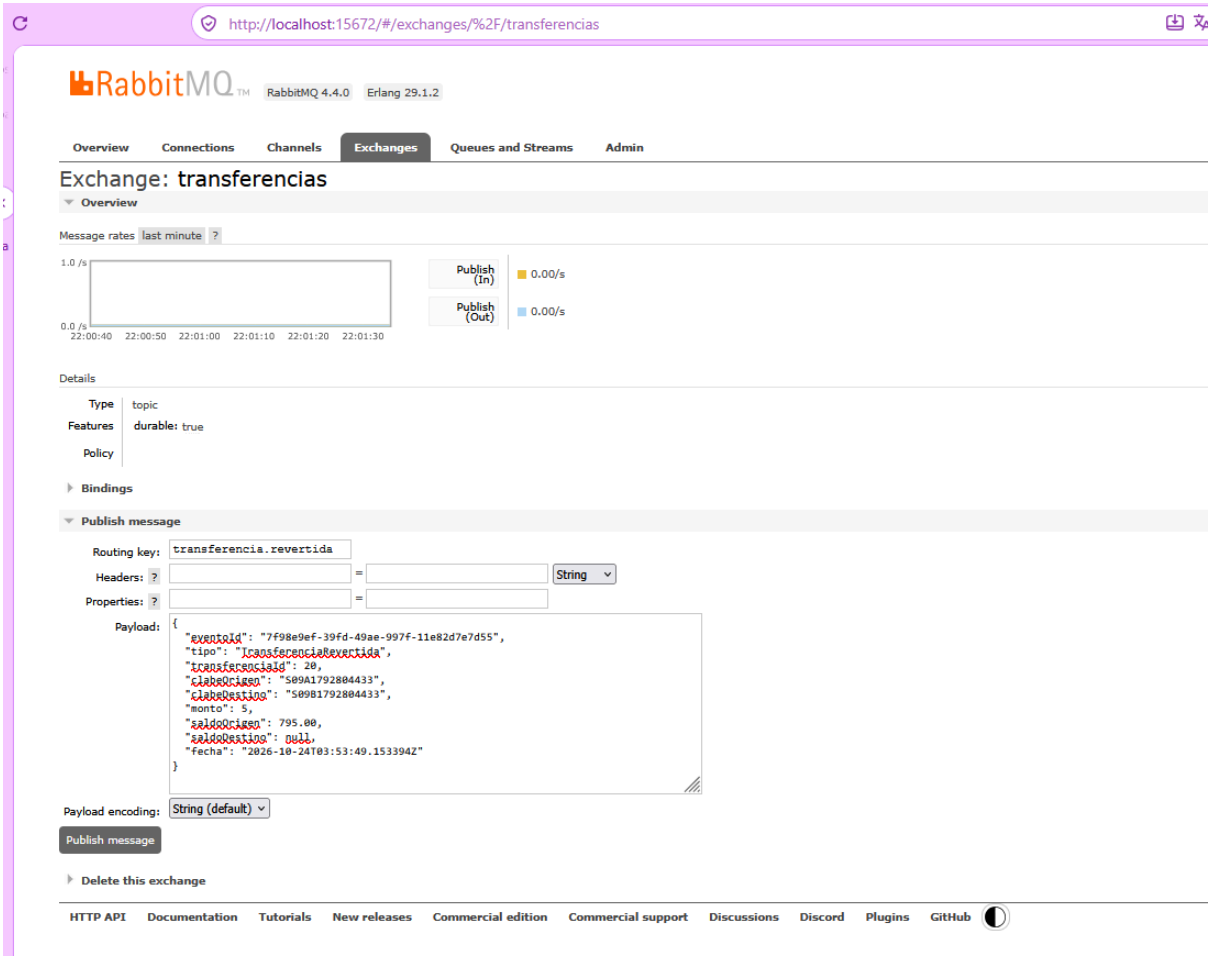


Primera entrega del evento

A las 21:55:10 comprobé que la transferencia 20 tenía el eventoId 7f98e9ef-39fd-49ae-997f-11e82d7e7d55. La bandeja mostraba un aviso de esa operación y la tabla evento_procesado contenía una fila con el mismo identificador. Esta es la referencia anterior a republicar el evento.



Volví a publicar el evento de la transferencia 20 usando el mismo eventoId. Al consultar las notificaciones a las 22:00:14, comprobé que el aviso seguía apareciendo una sola vez, con el mismo identificador y fecha. El log mostró que el consumidor reconoció el evento duplicado, lo ignoró y confirmó su recepción con un ACK. Esto demuestra que recibir nuevamente el mismo evento no genera otro aviso.




```
cecis@Asus_Ceci MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ date '+%F %T'
curl -i "http://localhost:8080/api/notificaciones?clabe=S09A1792804433"

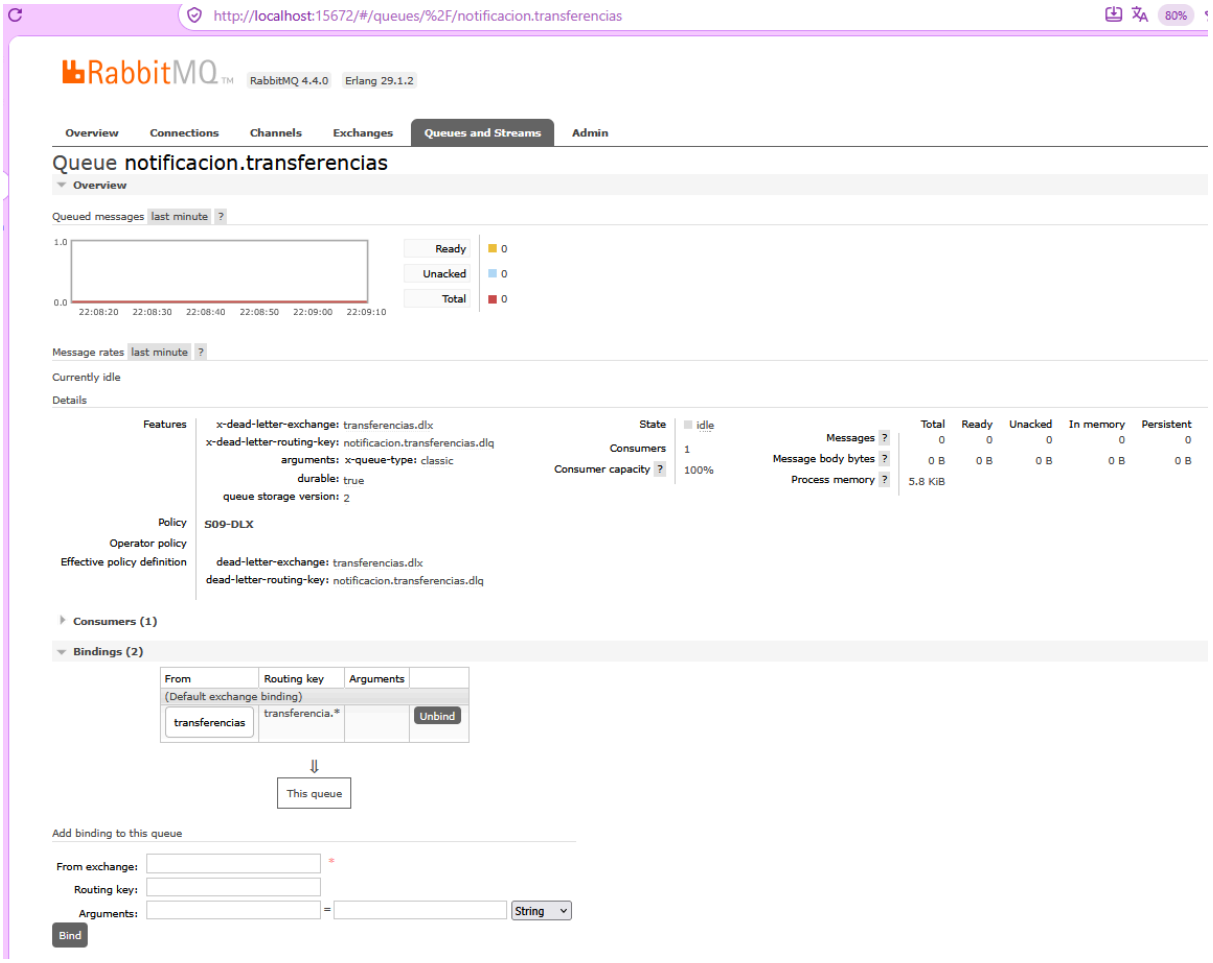
docker compose logs --since=5m notificacion | grep -E 'duplicado ignorado|ACK después'
2026-10-23 22:00:14
HTTP/1.1 200 OK
transfer-encoding: chunked
Content-Type: application/json
Date: Sat, 24 Oct 2026 04:00:15 GMT

[{"id":20,"clabeCuenta":"S09A1792804433","mensaje":"Tu transferencia 20 no se realizó y te devolvimos 5, saldo resultante 795.00","fecha":"2026-10-24T03:
53:49.358081Z"},{"id":19,"clabeCuenta":"S09A1792804433","mensaje":"Tu transferencia 19 no se realizó y te devolvimos 20.00, saldo resultante 795.00","fec
ha":"2026-10-24T03:36:47.348698Z"},{"id":18,"clabeCuenta":"S09A1792804433","mensaje":"Tu transferencia 19 no se realizó y te devolvimos 20.00, saldo resu
ltante 795.00","fecha":"2026-10-24T03:10:37.947602Z"},{"id":16,"clabeCuenta":"S09A1792804433","mensaje":"Transferencia 18 enviada por 10, saldo resultant
e 795.00","fecha":"2026-10-24T02:50:34.353905Z"},{"id":14,"clabeCuenta":"S09A1792804433","mensaje":"Transferencia 17 enviada por 10, saldo resultante 805
.00","fecha":"2026-10-24T02:50:34.332200Z"},{"id":12,"clabeCuenta":"S09A1792804433","mensaje":"Transferencia 16 enviada por 10, saldo resultante 815.00",
"fecha":"2026-10-24T02:50:33.856032Z"},{"id":10,"clabeCuenta":"S09A1792804433","mensaje":"TRANSFERENCIA_ENVIADA por 100, saldo resultante 900.00","fecha"
:"2026-10-24T01:15:25.938823Z"}]notificacion-1  | 2026-10-24T03:57:19.247Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.notificacion.NotificacionServ
ice      : Evento duplicado ignorado: eventoId=7f98e9ef-39fd-49ae-997f-11e82d7e7d55
notificacion-1  | 2026-10-24T03:57:19.251Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : ACK después de guardar
: eventoId=7f98e9ef-39fd-49ae-997f-11e82d7e7d55, transferenciaId=20
```

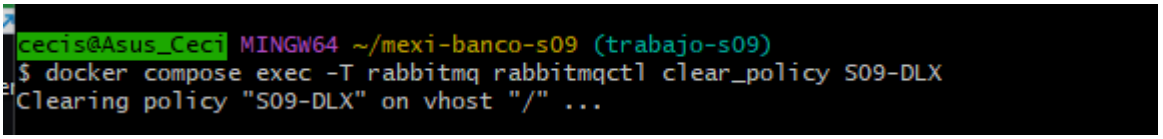
Dejé la configuración de mensajes rechazados en el código de la cola. El argumento x-dead-letter-exchange indica el exchange transferencias.dlx, y x-dead-letter-routing-key indica la clave notificacion.transferencias.dlq. Así, los mensajes rechazados sin reencolarlos pueden enviarse a la DLQ para revisarlos.

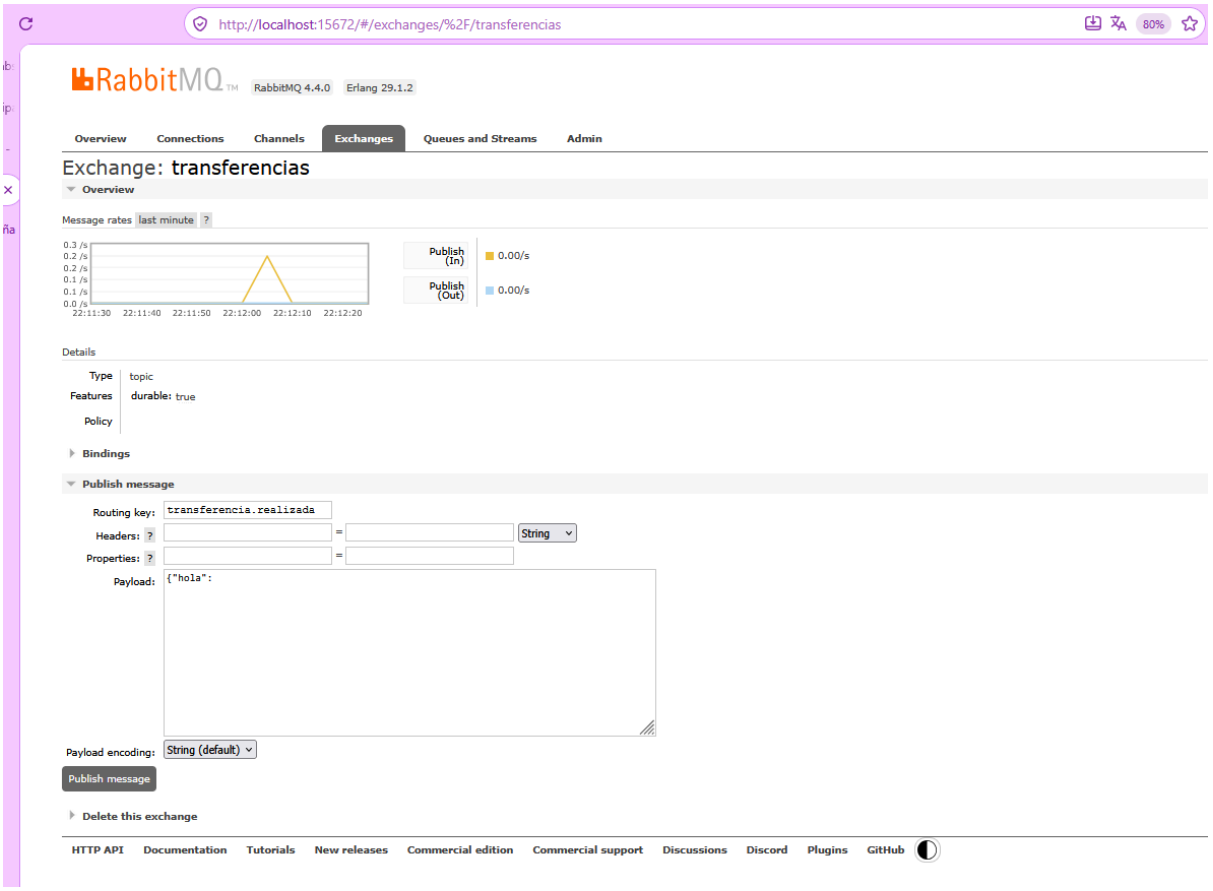
```
cecis@Asus_Ceci MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose up -d --no-deps notificacion
docker compose ps -a notificacion
[+] up 1/1
 ✔ Container mexi-banco-v06-notificacion-1 Started                                                                                                1.7s
NAME                              IMAGE                             COMMAND               SERVICE        CREATED          STATUS                                  PORT
S
mexi-banco-v06-notificacion-1   mexi-banco-v06-notificacion   "java -jar app.jar"   notificacion   5 seconds ago   Up 2 seconds (health: starting)   8080
/tcp

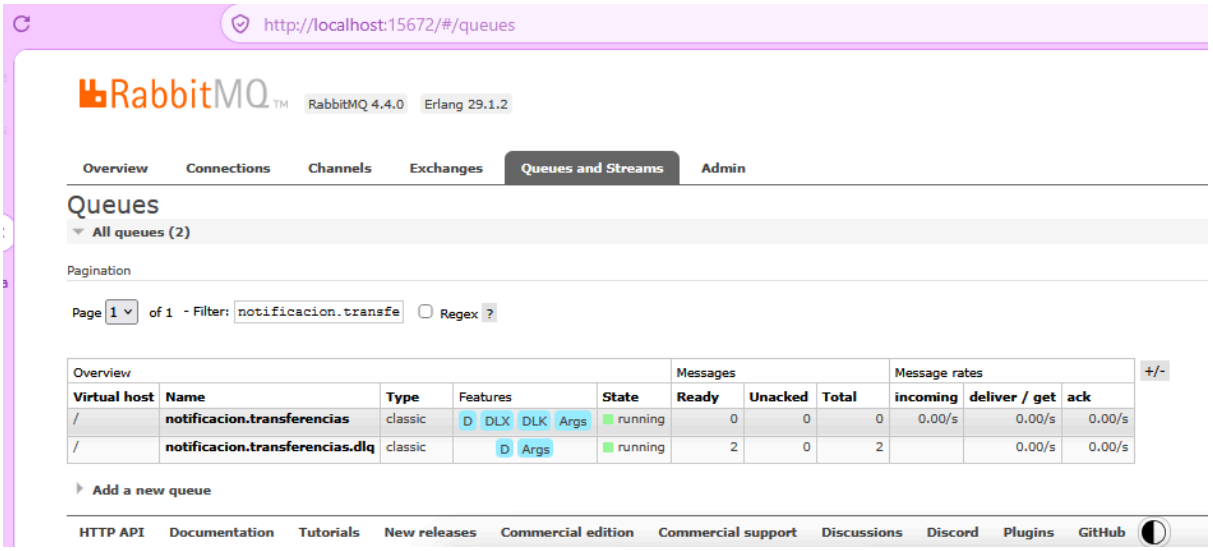
cecis@Asus_Ceci MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose ps -a notificacion
NAME                              IMAGE                             COMMAND               SERVICE        CREATED          STATUS                    PORTS
mexi-banco-v06-notificacion-1   mexi-banco-v06-notificacion   "java -jar app.jar"   notificacion   42 seconds ago   Up 40 seconds (healthy)   8080/tcp
```



Después de eliminar la política temporal, publiqué el mensaje incompleto `{"hola":` con la clave transferencia.realizada. La cola principal quedó vacía y la DLQ pasó de uno a dos mensajes. Esto muestra que el mensaje inválido fue apartado usando la configuración incluida en el código, sin quedarse en un ciclo de reintentos.



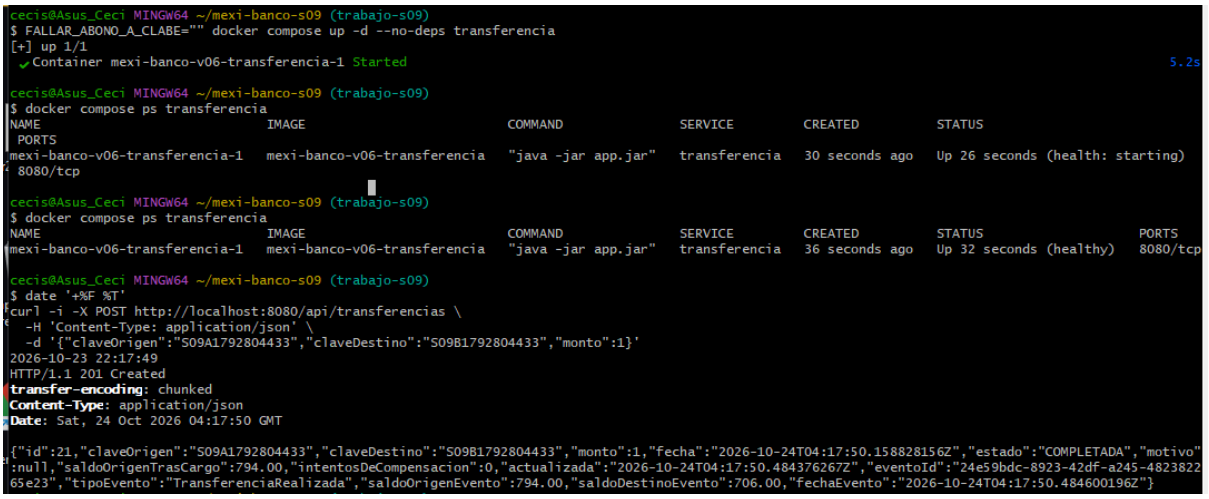




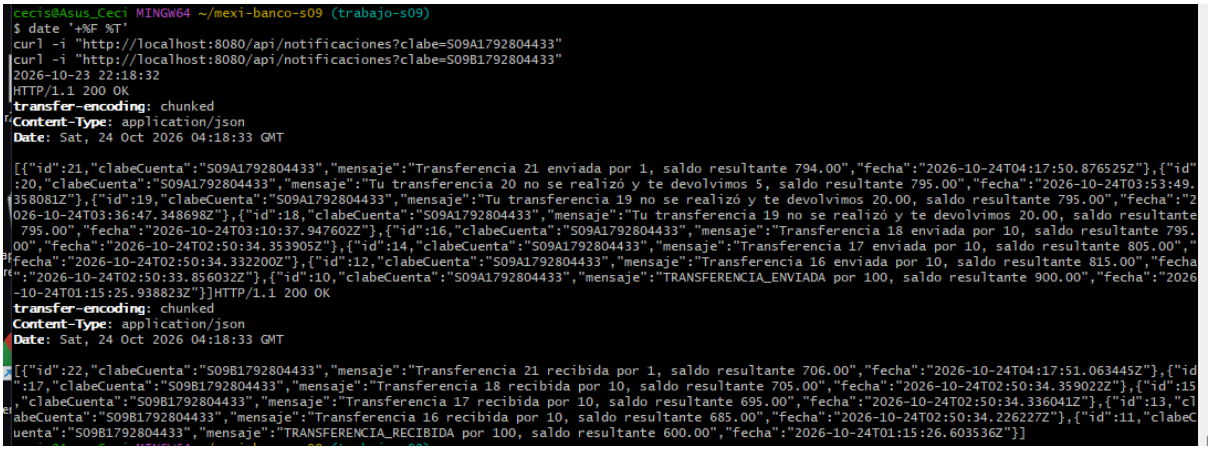
Al revisar el segundo mensaje de la DLQ, encontré el JSON incompleto que había publicado. El encabezado x-death indicó que fue rechazado desde la cola notificacion.transferencias y conservó la clave original transferencia.realizada. Así pude identificar el mensaje que causó el error sin eliminarlo.



A las 22:17:49 solicité una transferencia de 1 peso después de apartar el mensaje inválido en la DLQ. La operación 21 respondió 201 Created y quedó COMPLETADA, con saldo de origen de 794.00 y saldo de destino de 706.00.



A las 22:18:32 consulté las notificaciones de ambas cuentas. Encontré el aviso de envío de la transferencia 21 en origen y el de recepción en destino, con saldos de 794.00 y 706.00. Esto confirmó que el mensaje inválido quedó apartado en la DLQ y no bloqueó el procesamiento de los siguientes eventos.



¿Qué pasa si RabbitMQ está apagado cuando la transferencia se completa? ¿Los publisher confirms lo resuelven?

> En mi implementación, la transferencia puede quedar completada en la base de datos aunque falle la publicación del evento. Si RabbitMQ no está disponible, el aviso podría no llegar. Los publisher confirms permiten saber si el broker confirmó la publicación, pero no hacen que guardar la transferencia y enviar el evento sean una sola operación. Tampoco confirman que el consumidor haya

guardado el aviso. Para recuperar publicaciones pendientes de forma confiable se necesitaría implementar un outbox, que no incluí en esta versión.

¿Por qué hacer el ACK después de guardar y no antes?

Si confirmo el mensaje antes de guardar y el servicio falla, RabbitMQ ya lo considera atendido y podría perderse el aviso. Por eso hago el ACK después de confirmar la transacción en la base de datos. Aun así, si el servicio falla después de guardar pero antes del ACK, el mensaje puede llegar otra vez. Para ese caso guardé el eventoId junto con los avisos en la misma transacción, evitando que se generen duplicados.

¿Qué elegí al usar notificaciones asíncronas y qué sacrifico?

Elegí que las transferencias puedan continuar aunque el servicio de notificaciones esté apagado, siempre que RabbitMQ esté disponible para recibir los eventos. Los mensajes quedan en la cola y se procesan cuando el consumidor vuelve a funcionar. A cambio, el aviso puede tardar en aparecer: la transferencia y la notificación no se completan al mismo tiempo. Esto es un ejemplo de consistencia eventual.